

THE ULTIMATE

COMPUTER SCIENCE CURRICULUM

Full-Stack Engineer → Machine Learning Engineer

A complete, self-study degree replacement — built for the ambitious Filipino engineer. Version 3.0

6 Phases

27 Subjects

130+ Projects

~2,840 Hours

How to Use This Curriculum

This is a complete, self-directed CS degree — structured like the best university programs but rebuilt around what actually makes you hireable. Version 3 adds 7 new subjects, estimated hours per subject, star-rated resources, common mistakes callouts, phase capstone projects, and salary negotiation scripts specific to the Philippines.

Ph	Name	Covers	Hours	Outcome
1	Foundations	Python, Math, Linux, Git, TypeScript	520 hrs	Think like a programmer
2	Core CS Mastery	DSA, OOP, Databases, Networks, App Security	720 hrs	Think like a CS graduate
3	Full-Stack Eng.	Backend, Frontend, Cloud, System Design, DevOps	580 hrs	Build & ship products
4	ML & AI Eng.	ML, Deep Learning, MLOps, LLMs, Data Viz, Data Eng.	820 hrs	Become an ML Engineer
5	Career Prep	Interviews, Portfolio, Negotiation	200 hrs	Get hired at your target salary

Study Principles

- Build something every week — passive reading does not wire your brain. Code does.
- Struggle is the signal. If it feels easy, you are not at the edge of your growth.
- Solve 1 DSA problem every single day from Week 1. No exceptions. It compounds.
- Finish one resource completely before starting another. Tutorial hopping is procrastination.
- Teach what you learn — write blog posts, explain concepts out loud. It forces real understanding.
- At 5 focused hours/day, this curriculum takes about 570 days (~19 months). You have roughly that much college left.
- Consistency beats intensity: 2 focused hours daily beats 14 hours on Sunday every single time.
- The interview habit at each phase is not optional — it prevents starting cold at Phase 5.

When Stuck

When stuck for more than 30 minutes: (1) Re-read the error message — it almost always tells you the answer. (2) Rubber-duck debug: explain your code out loud to yourself or a pillow — this works. (3) Search the EXACT error message in quotes on Google. (4) Post on Stack Overflow or a Discord — write a minimal reproducible example first. (5) Walk away for 20 minutes. Your brain keeps working.

PH Context

Cost to complete this curriculum: P0 to P5,000 total. Paid items: LeetCode Premium (~\$35/mo, optional), some Udemy courses on sale (~P300), DDIA and Hands-On ML books (~P1,500 combined). Everything else is free. Total investment: your time. There is no excuse.

Daily & Weekly Study Schedule

Use this as your non-negotiable daily structure. Protect these blocks like your most important meeting.

Day	Block 1 (1-1.5 hr)	Block 2 (1.5-2 hr)	Block 3 (1-2 hr)
Mon	DSA Practice (1 LeetCode problem)	Subject Theory Study	Build / Project Work
Tue	DSA Practice (1 LeetCode problem)	Subject Theory Study	Build / Project Work
Wed	DSA Practice (1 LeetCode problem)	Review Notes + Active Recall	Mini project or coding exercise
Thu	DSA Practice (1 LeetCode problem)	Subject Theory Study	Build / Project Work
Fri	DSA Practice (1 LeetCode problem)	Subject Theory Study	Build / Project Work
Sat	2 LeetCode problems (harder)	Deep work — longer project session	Write blog post or README
Sun	Rest or light reading only	Plan next week: topics + goals	GitHub push — commit anything

Must Do

Target: 4-6 focused hours per day. 'Focused' means: phone in another room, website blocker on, Pomodoro timer running (25 min work / 5 min break). Track hours in a spreadsheet. Aim for 150+ hours per month during intense phases. At that pace, the entire curriculum takes about 19 months.

Visual Learning Roadmap & Dependency Map

Each phase depends on the previous. Subjects within a phase can partially overlap. Never start Phase N+1 before Phase N is 80% complete.

PHASE 1 — FOUNDATIONS (~520 hrs)	CS101 Python → CS105 TypeScript → CS102 Math (parallel) → CS103 OS/Linux → CS104 Git & Tooling
PHASE 2 — CORE CS (~720 hrs)	CS201 DSA (ongoing) → CS202 OOP & Patterns → CS203 Databases → CS204 Networks → CS205 App Security
PHASE 3 — FULL-STACK (~580 hrs)	CS301 Backend (FastAPI) → CS302 Frontend (React+TS) → CS305 Cloud (AWS/GCP) → CS303 System Design → CS304 DevOps
PHASE 4 — ML & AI (~820 hrs)	CS401 ML Foundations → CS402 Deep Learning → CS403 MLOps+LLM → CS404 Specialization → CS405 Data Viz → CS406 Data Eng.
PHASE 5 — CAREER (~200 hrs)	CS501 Technical Interviews (DSA + System Design) → CS502 Portfolio + Brand + Job Search + Negotiation

Pro Tip

Parallel tracks you CAN run simultaneously: CS102 Math with CS101 Python. CS104 Git from week 1 of everything. CS105 TypeScript after Week 4 of CS101. CS205 Security after CS203 and CS204 are 50% done. CS303 System Design while finishing CS301 Backend. CS405 Data Viz while doing CS401 ML. Never skip dependencies. Never start Phase 3 before Phase 2 is 80% done.

What's New in Version 3

Addition	Location	Why It Matters
CS105 TypeScript & Type-Safe Development (NEW)	Phase 1	Type safety from day one — required for every serious job in 2025
CS205 Application Security (NEW)	Phase 2	OWASP Top 10, secure coding, PortSwigger labs — fintech roles test this
CS305 Cloud Fundamentals AWS/GCP (NEW)	Phase 3	AWS/GCP listed in majority of PH mid-level job postings
CS405 Data Visualization & Communication (NEW)	Phase 4	Half the ML job is communicating results — SHAP, Streamlit, Plotly
CS406 Data Engineering Foundations (NEW)	Phase 4	Many 'ML Engineer' PH roles are data engineering — know both
Estimated Hours Per Subject (NEW)	All Phases	Now shows ~hours per subject so you can plan realistically
Star-Rated Resources (NEW)	All Phases	1-5 stars with honest opinions — prioritize 5-star resources first
Common Mistakes Callouts (NEW)	All Subjects	What 90% of self-learners get wrong — specific and actionable
Phase Capstone Projects (NEW)	All Phases	One integrating project per phase — portfolio-ready, production-quality
Interview Habits per Phase (NEW)	All Phases	Mock interview cadence from Phase 2 — not starting cold at Phase 5
Salary Negotiation Scripts (NEW)	Phase 5 CS502	PH-specific negotiation language — word-for-word scripts included
Total curriculum hours: ~2,840 (UPDATED)	Overview	At 5 focused hrs/day: ~19 months. At 3 hrs/day: ~32 months

PHASE 1

~520 hrs

Foundations

Build the bedrock. Every concept in this curriculum depends on what you learn here.

Est. Duration	4–6 months	Est. Total Hours	~520 hours
---------------	------------	------------------	------------

Must Complete (MVP)	Nice-to-Have (If Time Allows)
• CS101 Python (all 8 weeks)	• CS102 Calculus (delay to Phase 4)
• CS102 Math Part 1 & 3	• CS103 deep OS internals
• CS104 Git & Tooling	
• CS105 TypeScript basics	

Interview Habit

From Week 1: solve 1 easy LeetCode problem every day. Don't skip. Even if you only understand the solution after — that's fine. Log it.

CS101	Programming Fundamentals with Python	Free
Duration	8 weeks	Beginner
Est. Hours	~160 focused hours	
Prereqs	None	

Python is your first real weapon. Don't rush this. Most self-taught developers skip fundamentals and spend the next 2 years patching holes. Every exercise matters.

Curriculum Breakdown

Weeks 1-2: Python Basics

- Variables, data types: int, float, str, bool, None
- Arithmetic, comparison, logical, bitwise operators
- print(), input(), f-strings, format()
- Type casting and type checking
- PEP 8 style guide — write clean code from day one

Week 3: Control Flow

- if/elif/else — conditionals
- while loops — condition-based iteration
- for loops — iterating over sequences
- break, continue, pass
- range() and loop patterns — very common in interviews

Week 4: Functions

- Defining and calling functions
- Parameters, default arguments, *args, **kwargs
- Return values
- Scope: local vs global

- Recursion: base case + recursive case
- Lambda, map(), filter()

Week 5: Data Structures

- Lists: indexing, slicing, list comprehensions
- Tuples: immutability, packing/unpacking
- Dictionaries: key-value, nested, dict comprehensions
- Sets: union, intersection, difference
- When to use which — this question comes up in interviews

Week 6: OOP

- Classes and objects
- Instance attributes, class attributes, methods
- `__init__`, `__str__`, `__repr__`
- Encapsulation, Inheritance (`super()`), Polymorphism
- Abstract base classes (abc module)

Week 7: File I/O & Error Handling

- Reading/writing text files
- CSV, JSON handling
- try/except/else/finally — never let programs crash silently
- Raising custom exceptions
- Context managers: with statement

Week 8: Modules & Standard Library

- import, from, as
- os, sys, pathlib
- math, random, datetime
- collections: Counter, defaultdict, deque
- Creating your own modules and packages

Common Mistakes — 90% of self-learners get these wrong

Skipping exercises and just reading — reading is not learning, building is

Not using a virtual environment from day one

Ignoring PEP 8 — bad habits formed here are hard to break later

Copy-pasting solutions without understanding them line by line

Practice Projects

Project	Description	Lvl	Hireable Version Has...
CLI Calculator	Terminal calculator with full error handling	Beg	Handles divide-by-zero, invalid input, history of last 10 calculations saved to file
Contact Book	Add/search/delete contacts saved to JSON	Beg	Search by partial name, validates phone format, handles missing file gracefully
Grade Tracker	Store student grades, compute averages, letter grades	Beg	Multiple subjects, GWA computation, export to CSV, full CLI menu
Number Guessing Game	Binary-search-style guessing with score history	Beg	Tracks best/worst score, average guesses per session, difficulty levels

Self-Check — Can you answer these?

- ☐ Can you write a class with inheritance without looking it up?
- ☐ Can you read a JSON file, modify it, and write it back?
- ☐ Can you explain the difference between a list and a tuple?

- Can you write a recursive function with a base case and test it?
- Can you explain what a decorator does before you've studied them formally?

When Stuck

Stuck on OOP: forget classes for a day. Write 10 small functions that work together. Then ask: how do I group this data and behavior? That group is a class.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Automate the Boring Stuff	Book	Free	★★★★★	automatetheboringstuff.com — best free Python intro, do every exercise
CS50P Harvard	Course	Free	★★★★★	Best produced free Python course, high quality video + exercises
Python Crash Course	Book	Paid	★★★★★	Eric Matthes — most beginner-friendly printed Python book
HackerRank Python	Practice	Free	★★★★★	Complete all Easy problems in the Python domain — good drill

CS102	Mathematics for Computer Science	Free
Duration	10 weeks (parallel)	Intermediate
Est. Hours	~120 focused hours	
Prereqs	High school algebra	

You said you have good logic. This is where it proves itself. Math is not memorization — it is precise reasoning. Focus on intuition first, formulas second.

Curriculum Breakdown

Part 1: Discrete Math

- Logic: propositions, truth tables, logical equivalence
- Proof techniques: direct, contradiction, mathematical induction
- Set theory: sets, subsets, power sets, Cartesian products
- Relations and functions: injective, surjective, bijective
- Sequences, summations, Big-O notation intro

Part 2: Combinatorics & Probability

- Counting: rule of sum, rule of product
- Permutations and combinations
- Pigeonhole principle — surprisingly useful in interviews
- Basic probability, conditional probability
- Bayes' theorem — essential for ML
- Expected value and variance

Part 3: Linear Algebra (Critical for ML)

- Vectors: dot product, magnitude, unit vectors, projections
- Matrices: add, multiply, transpose, inverse
- Determinants and geometric meaning
- Linear transformations
- Eigenvalues and eigenvectors — core to PCA, PageRank
- Systems of equations: Gaussian elimination

Part 4: Calculus for ML

- Derivatives: rules, chain rule
- Partial derivatives — functions of multiple variables
- Gradient: direction of steepest ascent
- Gradient descent — the optimizer behind every neural network
- Integration basics, probability distributions (normal, uniform, Bernoulli)

Part 5: Graph Theory

- Graphs: vertices, edges, directed vs undirected
- Adjacency matrix and adjacency list
- BFS and DFS (preview of DSA)
- Trees, spanning trees, minimum spanning trees
- Applications: maps, compilers, social networks, dependency resolution

Common Mistakes — 90% of self-learners get these wrong

Treating math as memorization — understand WHY each formula is true

Skipping proofs — proofs train the exact logical thinking CS needs

Not connecting math to code — always implement what you learn in numpy

Doing Part 4 (Calculus) before the ML phase — delay it until you need it

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Gaussian Elimination	Implement from scratch in Python	Int	Handles any $n \times n$ matrix, detects no-solution and infinite-solutions cases
Gradient Descent Visualizer	Plot gradient descent on a 2D function	Int	Shows learning rate effects, convergence, animation with matplotlib
Truth Table Generator	Generate truth tables for any logical formula	Int	Parses expression string, handles AND/OR/NOT/XOR/IMPLIES operators
Probability Simulator	Monte Carlo simulation of dice/coin/card problems	Beg	Compares simulated vs theoretical probability across 1M trials

Self-Check — Can you answer these?

- ☐ Can you explain what an eigenvector is geometrically — no formula?
- ☐ Can you derive gradient descent from first principles on paper?
- ☐ Can you prove by induction that $\sum 1..n = n(n+1)/2$?
- ☐ Can you explain Bayes' theorem with a spam filter example?
- ☐ Can you compute a matrix inverse by hand for a 2×2 matrix?

When Stuck

If linear algebra feels abstract: open numpy and experiment. `np.dot()`, `np.linalg.eig()`. Seeing real numbers transform makes the theory click immediately.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
MIT 6.042J	Course	Free	★★★★★	Math for CS on MIT OCW — includes problem sets, best free resource
3Blue1Brown Linear Algebra	Video	Free	★★★★★	Essence of Linear Algebra — best visual intuition available anywhere

Math for ML (Deisenroth)	Book	Free	★★★★★	Free PDF, directly tied to ML — use as your reference
Khan Academy Calculus	Course	Free	★★★★★	Multivariable calculus — go at your own pace, good for Part 4

CS103	Computer Architecture & Operating Systems	Free
Duration	6 weeks	Intermediate
Est. Hours	~80 focused hours	
Prereqs	CS101	

To write fast code, you must understand the machine it runs on. This answers: why is Python slow? What actually happens when you type `python script.py`? Why does RAM size matter?

Curriculum Breakdown

Part 1: How Computers Work

- Binary, hexadecimal, two's complement arithmetic
- Boolean logic: AND, OR, NOT, XOR — these ARE the computer
- CPU: ALU, control unit, registers, clock cycles
- Memory hierarchy: registers > L1/L2 cache > RAM > disk — speeds matter
- Von Neumann architecture
- Instruction cycle: fetch, decode, execute

Part 2: OS Fundamentals

- What an OS does: abstraction and resource management
- Processes vs threads — critical for ML serving and backend
- Process lifecycle and scheduling: FCFS, SJF, Round Robin
- Concurrency: race conditions, deadlocks — how to spot them
- Synchronization: mutex, semaphores, condition variables
- Memory management: paging, virtual memory, page faults
- File systems: inodes, directories, FAT vs ext4 vs APFS

Part 3: Linux Mastery

- Navigation: `ls`, `cd`, `pwd`, `find`, `locate`, `tree`
- File ops: `cp`, `mv`, `rm`, `mkdir`, `touch`, `ln`, `chmod`, `chown`
- Process control: `ps`, `top`, `htop`, `kill`, `jobs`, `&`, `nohup`
- Text processing: `grep`, `awk`, `sed`, `cut`, `sort`, `uniq`, `wc`, `tr`
- Pipes and redirection: `|`, `>`, `>>`, `<`, `2>&1`, `tee`
- Shell scripting: variables, loops, `if/elif`, functions, `$?`, `set -e`
- SSH, SCP, `rsync`, `tmux`, `screen` — essential for cloud server work
- `cron`: scheduling automated tasks

Common Mistakes — 90% of self-learners get these wrong

- Reading about Linux without actually using it — you must practice in a real terminal
- Not setting up WSL2 or a VM — there is no substitute for a real Linux environment
- Memorizing commands without understanding what they do
- Skipping the shell scripting section — every DevOps and MLOps role needs this

Practice Projects

Project	Description	Lvl	Hireable Version Has...
---------	-------------	-----	-------------------------

Backup Shell Script	Auto-backup a folder with timestamp and log	Int	Handles errors, sends desktop notification, runs via cron, rotates old backups
Process Scheduler Sim	Simulate Round Robin + SJF in Python	Int	Visual Gantt chart output, compare wait times across algorithms
/proc Explorer	Python system monitor reading from /proc	Int	Displays CPU, memory, top processes — refreshes every second

Self-Check — Can you answer these?

- ☐ Can you explain what happens between typing a URL and seeing a webpage (OS layer only)?
- ☐ Can you write a shell script with a loop, conditional, and function from memory?
- ☐ Can you explain virtual memory to a 10-year-old?
- ☐ Can you describe a deadlock with a real-world analogy?
- ☐ Can you find the PID of a process and kill it from the command line?

When Stuck

Install Ubuntu on VirtualBox or enable WSL2 on Windows. Every Linux topic must be practiced in a real terminal. Reading commands without typing them is useless.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
The Linux Command Line	Book	Free	★★★★★	shotts.us — free online, the definitive guide, do every exercise
OSTep (ostep.org)	Book	Free	★★★★★	Operating Systems: Three Easy Pieces — free PDF, best OS textbook
Nand2Tetris Part 1	Course	Free	★★★★★	Build a computer from NAND gates — mindblowing, fully free
CS:APP (Bryant)	Book	Paid	★★★★★	CMU's Computer Systems — most thorough resource, use as reference

CS104	Version Control & Developer Tooling	Free
Duration	2 weeks	Beginner
Est. Hours	~30 focused hours	
Prereqs	CS101	

Git is used every single day in every software job on the planet. Start using it on every project you build from this moment forward. Your GitHub commit history is part of your resume.

Curriculum Breakdown

Git Essentials

- init, clone, status, add, commit, push, pull, fetch
- Branches: create, switch, merge, delete, rename
- Merge conflicts: understanding, resolving, preventing — don't fear them
- git log, diff, stash, reset --soft/--hard, revert, clean
- git reflog — your time machine for recovering anything
- Rebase vs merge: interactive rebase, squash commits
- Tags and semantic releases

GitHub & Professional Workflow

- .gitignore — create this before your first commit, always
- Pull requests: writing good PR descriptions, requesting reviews
- Code review: giving and receiving feedback professionally
- GitHub Actions intro: CI/CD preview, running tests on push
- Conventional commits: feat:, fix:, docs:, chore:, refactor:, test:
- Semantic versioning: MAJOR.MINOR.PATCH and why it matters
- GitHub Projects: basic kanban for solo and team work

Dev Environment Setup

- VS Code: Pylance, GitLens, Prettier, ESLint, Docker, REST Client
- Python: venv, pip, requirements.txt, pip freeze, pip-tools
- pyproject.toml and modern Python packaging
- Linters: flake8, pylint, ruff (fastest)
- Formatters: black (Python), prettier (JS/TS)
- Pre-commit hooks: auto-format and lint before every commit
- Debugging: breakpoints, watch expressions, call stack, conditional breakpoints

Common Mistakes — 90% of self-learners get these wrong

Committing directly to main — use branches for everything from day one

Writing vague commit messages like 'fix stuff' or 'update'

Not using .gitignore — committing secrets, venv folders, __pycache__

Treating GitHub as a backup service rather than a collaboration tool

Practice Projects

Project	Description	Lvl	Hireable Version Has...
GitHub Portfolio Setup	Push all Phase 1 projects to GitHub with proper READMEs	Beg	Each repo: clear README with screenshots, .gitignore, descriptive commits, live demo link
Conflict Resolution	Simulate and fully resolve a 3-way merge conflict	Beg	Two branches with conflicting changes, resolved cleanly, documented in README

Self-Check — Can you answer these?

- ☐ Can you recover a deleted commit using git reflog?
- ☐ Can you explain the difference between git rebase and git merge and when to use each?
- ☐ Can you set up a Python virtual environment, install packages, and freeze dependencies?
- ☐ Can you write a pre-commit hook that runs black before every commit?

When Stuck

If you break a git repo: search the EXACT error message. Almost every git problem has a Stack Overflow answer. git reflog shows every action — you can always recover.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
learngitbranching.js.org	Interactive	Free	★★★★★	Gamified Git — complete every level, best interactive Git learning
Pro Git Book	Book	Free	★★★★★	git-scm.com/book — official, comprehensive, free, read chapters 1-5
Oh My Git!	Game	Free	★★★★★	Visual card game for Git — fun way to reinforce concepts

Conventional Commits	Docs	Free	★★★★ ■	conventionalcommits.org — the spec, short and worth reading fully
----------------------	------	------	-----------	---

CS105	TypeScript & Type-Safe Development	Free
Duration	3 weeks	Beginner
Est. Hours	~45 focused hours	
Prereqs	CS101	

TypeScript is no longer optional. Every serious frontend and backend role in 2025 expects it. Learning it now — before JavaScript habits are set — is far easier than retrofitting it later. Think of it as Python's type hints but for JavaScript.

Curriculum Breakdown

TypeScript Fundamentals

- Why TypeScript: catches entire categories of bugs before runtime
- Installing TypeScript: tsc, ts-node, tsconfig.json basics
- Primitive types: string, number, boolean, null, undefined, symbol
- Type annotations: explicit vs inferred — TypeScript is smart
- Arrays and tuples: string[], [string, number]
- Objects and type aliases: type User = { name: string; age: number }
- Union types: string | number — a value can be one of several types
- Intersection types: combining types with &
- Literal types: type Direction = 'left' | 'right' | 'up' | 'down'

Intermediate TypeScript

- Interfaces vs type aliases — when to use each
- Optional properties: name?: string
- Readonly: readonly id: number — prevents mutation
- Generics: function identity(x: T): T — write once, use for any type
- Generic constraints:
- Utility types: Partial, Required, Pick, Omit, Record
- Enums: const enums for performance
- Type narrowing: typeof, instanceof, in operator, discriminated unions
- Never type: exhaustive type checking

TypeScript in Practice

- Strict mode: always enable it — 'strict': true in tsconfig
- Type assertions: as Type — use sparingly
- Type guards: custom is Type functions
- Module system: import/export with types
- Declaration files: .d.ts — how to type third-party libraries
- tsconfig.json: target, lib, paths, baseUrl
- ESLint with TypeScript: @typescript-eslint/recommended
- Migrating JS to TS: --allowJs, gradual migration strategy

Common Mistakes — 90% of self-learners get these wrong

- Using any everywhere — it defeats the entire purpose of TypeScript
- Ignoring TypeScript errors with @ts-ignore instead of fixing them
- Not enabling strict mode — it catches the bugs that matter most
- Learning TS without learning JS first — understand the runtime first

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Type-Safe CLI Tool	Rewrite CS101 grade tracker in TypeScript	Beg	Full strict mode, no any types, proper interfaces for all data shapes
TS Utility Library	Build a typed utility library with generics	Int	10+ generic functions, 100% type coverage, published as npm package

Self-Check — Can you answer these?

- ☐ Can you explain the difference between an interface and a type alias?
- ☐ Can you write a generic function that works on any array type?
- ☐ Can you use discriminated unions to handle different response shapes?
- ☐ Can you configure tsconfig.json strict mode and explain what each option does?
- ☐ Can you narrow a union type using a custom type guard?

When Stuck

If a TypeScript error confuses you: copy the exact error into the TypeScript playground (typescriptlang.org/play) and simplify until you understand it. The playground shows types live.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
TypeScript Handbook	Docs	Free	★★★★★	typescriptlang.org/docs — official, comprehensive, read the whole thing
Total TypeScript (Pocock)	Course	Free/Paid	★★★★★	Best TypeScript course — free beginner section is excellent
TypeScript Deep Dive	Book	Free	★★★★★	basarat.gitbook.io — free, thorough, practical examples
Execute Program TS	Interactive	Paid	★★★★★	Spaced repetition TypeScript exercises — very effective learning

Phase Capstone

Build a fully type-safe Python + TypeScript CLI productivity app: a personal study tracker that logs your daily study sessions, LeetCode problems solved, and generates a weekly report. Backend: Python with type hints and Pydantic. CLI: TypeScript with strict mode. Data: JSON files with versioned schema. Push to GitHub with a professional README, commit history, and automated backup via shell script. This is your first portfolio piece.

PHASE 2

~720 hrs

Core CS Mastery

This is where real engineers are forged. Difficulty spikes here. Embrace the struggle.

Est. Duration	7–9 months	Est. Total Hours	~720 hours
----------------------	------------	-------------------------	-------------------

Must Complete (MVP)	Nice-to-Have (If Time Allows)
• CS201 DSA (all modules — no shortcuts)	• CS204 full network stack — understand TCP/HTTP, skip deep theory initially
• CS203 SQL & databases	
• CS202 OOP Design Patterns	
• CS205 Application Security basics	

Interview Habit

From Phase 2: do 1 Codeforces contest per week. Start with Div 3. Track your rating. Also: do one 30-min mock interview with a friend each month — just talking through a problem out loud.

CS201	Data Structures & Algorithms	Free
Duration	16 weeks + daily practice forever	Hard
Est. Hours	~320 focused hours	
Prereqs	CS101, CS102	

DSA is the most important subject in this entire curriculum. Every major tech company interviews on this. You can already solve 800 on Codeforces. By the end of this, target 1400+. The secret is pattern recognition — once you see the pattern, you can solve dozens of problems that look completely different on the surface.

Curriculum Breakdown

Module 1: Complexity Analysis

- Big-O: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$
- Big-Omega and Big-Theta — lower and tight bounds
- Time complexity vs space complexity — always analyze both
- Best / average / worst case — interviewers ask this
- Analyzing loops, nested loops, recursion trees
- Amortized analysis: dynamic array resizing example

Module 2: Arrays & Strings

- Static vs dynamic arrays, memory layout in RAM
- Two-pointer technique — one of the 10 patterns you must master
- Sliding window — fixed size and variable size variants
- Prefix sums and difference arrays — $O(n)$ range queries
- Kadane's algorithm — max subarray in $O(n)$
- String matching: naive $O(nm)$, KMP $O(n+m)$
- Rabin-Karp rolling hash for pattern matching

Module 3: Linked Lists

- Singly, doubly, circular — implement all from scratch
- Fast/slow pointer — Floyd's cycle detection algorithm
- Reversing iteratively and recursively — both ways
- Merging two sorted linked lists — classic interview problem
- Finding middle node, kth from end in one pass

Module 4: Stacks & Queues

- Stack LIFO: push/pop/peek — array and linked list implementations
- Queue FIFO: enqueue/dequeue — circular array implementation
- Deque: double-ended queue — used in sliding window maximum
- Monotonic stack — CRUCIAL: next greater element, histogram
- Priority Queue / Heap: insert $O(\log n)$, extract-min $O(\log n)$
- Applications: expression evaluation, BFS, undo/redo, task scheduling

Module 5: Trees

- Binary trees: height, diameter, balance, complete vs full vs perfect
- Traversals: inorder, preorder, postorder, level-order (BFS)
- Binary Search Tree: insert $O(h)$, search $O(h)$, delete $O(h)$
- BST balancing concepts: AVL rotations, Red-Black tree properties
- Heaps: max-heap, min-heap, heapify $O(n)$, heap sort $O(n \log n)$
- Trie: prefix tree — autocomplete, word search, IP routing
- Segment tree: range sum/min/max queries $O(\log n)$

Module 6: Hash Tables

- Hash functions: uniform distribution, avalanche effect
- Collision resolution: chaining (linked lists) vs open addressing (probing)
- Load factor: when to resize, why 0.75 is common threshold
- HashMap vs HashSet: when each is appropriate
- Two-sum pattern: the most common interview question family
- Rolling hash: Rabin-Karp, substring matching

Module 7: Graphs

- Representations: adjacency matrix $O(V^2)$ vs adjacency list $O(V+E)$
- BFS: shortest path in unweighted graph, level-order traversal
- DFS: connected components, cycle detection, topological sort
- Dijkstra: shortest path weighted graph, priority queue, $O((V+E) \log V)$
- Bellman-Ford: handles negative weights, detects negative cycles $O(VE)$
- Floyd-Warshall: all-pairs shortest path $O(V^3)$
- Kruskal's MST: sort edges + union-find $O(E \log E)$
- Prim's MST: greedy + priority queue $O((V+E) \log V)$
- Union-Find (DSU): path compression + union by rank — nearly $O(1)$

Module 8: Sorting & Searching

- Bubble, selection, insertion: $O(n^2)$ — know when each is actually useful
- Merge sort: stable, $O(n \log n)$, extra space — the divide-and-conquer archetype
- Quick sort: in-place, $O(n \log n)$ average, $O(n^2)$ worst — how to avoid worst case
- Heap sort: in-place, $O(n \log n)$, not stable
- Counting sort: $O(n+k)$, non-comparison, integers only
- Radix sort: $O(nk)$, stable, faster than comparison sort for integers
- Binary search: 6 variants — find first/last occurrence, rotated array, answer-search

Module 9: Dynamic Programming

- DP definition: overlapping subproblems + optimal substructure
- Step 1: identify the decision at each step
- Step 2: define state — what information do you need to make the decision?
- Step 3: write recurrence relation — how does current state depend on previous?
- Step 4: memoization (top-down) or tabulation (bottom-up)

- Classic 1D DP: Fibonacci, climbing stairs, house robber, coin change
- Classic 2D DP: grid paths, LCS, LIS, edit distance, knapsack
- DP on trees: diameter, path sum, subtree problems
- Interval DP: matrix chain multiplication, burst balloons
- Backtracking: N-Queens, subsets, permutations, combination sum, sudoku

Common Mistakes — 90% of self-learners get these wrong

Solving the same type of problem 50 times instead of 50 different patterns

Looking at the solution after 5 minutes instead of struggling for 30

Not analyzing time/space complexity after every problem — this is mandatory

Grinding LeetCode without understanding patterns — patterns are the whole point

Not doing timed practice — speed matters in real interviews

Practice Projects

Project	Description	Lvl	Hireable Version Has...
DSA Library	Implement every data structure from scratch	Adv	Clean API, docstrings, time/space complexity in docs, full pytest test suite
LeetCode NeetCode 150	Complete the NeetCode 150 problem set	Adv	All solved, patterns labeled, time/space complexity documented for each
Codeforces 100+	Solve 100+ problems rated 800-1300	Adv	At least 30 at 1200+ rating, participate in 10+ Div 3 contests
DP Visualizer	Terminal visualizer for DP table filling	Adv	Supports knapsack, LCS, edit distance — shows table step by step with colors

Self-Check — Can you answer these?

- ☐ Can you implement a heap from scratch, including heapify, in under 20 minutes?
- ☐ Can you solve an unseen DP problem by deriving the recurrence relation from scratch?
- ☐ Can you explain Dijkstra vs Bellman-Ford — when each fails and why?
- ☐ Can you trace BFS and DFS on a graph with 8 nodes by hand in under 3 minutes?
- ☐ Can you solve a sliding window problem you've never seen before in under 15 minutes?
- ☐ Can you explain union-find with path compression and why it's nearly $O(1)$?

When Stuck

Stuck on DP? Step back entirely. Ask: what decision am I making at each step? What information do I need to remember to make that decision? That information IS your state. Write the recurrence before writing any code.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
neetcode.io	Platform	Free	★★★★★	Best organized roadmap + video explanations — use as your primary resource
MIT 6.006 (YouTube)	Course	Free	★★★★★	Best algorithmic thinking course — watch lectures, do problem sets
CP Handbook (Laaksonen)	Book	Free	★★★★★	Competitive Programmer's Handbook — free PDF, best CP reference

CLRS (Algorithms)	Book	Paid	★★★★ ■	The bible of DSA — use as reference, not cover to cover
Abdul Bari Algorithms	Video	Free	★★★★ ■	YouTube — excellent intuitive explanations, good supplement

CS202	Object-Oriented Design & Design Patterns	Free
Duration	6 weeks	Intermediate
Est. Hours	~70 focused hours	
Prereqs	CS101	

Knowing how to code is not enough. Knowing how to DESIGN code is what separates junior from senior engineers. This subject teaches you to write code that other people can understand, safely extend, and not accidentally break.

Curriculum Breakdown

SOLID Principles

- S: Single Responsibility — one class, one reason to change — not 'one method'
- O: Open/Closed — open for extension, closed for modification — use abstractions
- L: Liskov Substitution — subclasses must be substitutable without breaking behavior
- I: Interface Segregation — don't force classes to implement methods they don't need
- D: Dependency Inversion — depend on abstractions, not concretions — enables testing
- Refactoring exercise: take a 300-line god class and apply SOLID step by step

Creational Patterns

- Singleton: only one instance — DB connection pool, config manager — use sparingly
- Factory Method: delegate object creation to subclasses — decouples creation from use
- Abstract Factory: create families of related objects without specifying concrete classes
- Builder: construct complex objects step by step — separates construction from representation
- Prototype: clone an existing object — useful when creation is expensive

Structural Patterns

- Adapter: make incompatible interfaces work together — legacy code integration
- Decorator: add behavior dynamically without inheritance — Python @decorators are this
- Facade: simplified interface to a complex subsystem — hide complexity from callers
- Proxy: control access to an object — lazy loading, caching, access control
- Composite: treat individual objects and compositions uniformly — tree structures

Behavioral Patterns

- Observer: event system — subscribe and notify — the basis of event-driven systems
- Strategy: swap algorithms at runtime — sort by different criteria, payment methods
- Command: encapsulate a request as an object — undo/redo, job queues, macros
- Iterator: sequential access without exposing internals — Python's `__iter__` is this
- State: change behavior when internal state changes — traffic lights, order status
- Template Method: define the skeleton of an algorithm, let subclasses fill details

UML & Code Modeling

- Class diagrams: classes, attributes, methods, relationships (association, aggregation, composition, inheritance)
- Sequence diagrams: how objects interact over time — method call flows
- When to draw diagrams vs when to just code — don't over-engineer
- Reading UML in job descriptions, system design docs, and onboarding materials

Common Mistakes — 90% of self-learners get these wrong

Applying patterns everywhere — patterns solve specific problems, not all problems

Learning pattern names without understanding the problem they solve

Using inheritance when composition is more appropriate — prefer composition

Thinking SOLID is rules to memorize rather than principles to internalize

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Refactor Grade Tracker	Apply SOLID to CS101 grade tracker	Int	Can add new grade types and export formats without touching existing classes
Plugin System	Extensible system using Strategy + Factory	Int	New plugins drop in as files — no core code modified, demonstrates OCP
Event System	Full pub/sub notification system	Int	Multiple listeners, event filtering, async support, dead letter queue for failures

Self-Check — Can you answer these?

- ☐ Can you identify which SOLID principle a piece of code violates and explain why?
- ☐ Can you implement the Observer pattern from memory in under 15 minutes?
- ☐ Can you explain why Singleton is often called an anti-pattern in testable code?
- ☐ Can you draw a class diagram for a simple ride-sharing system from scratch?
- ☐ Can you explain the difference between Decorator and Inheritance with a concrete example?

When Stuck

Patterns are vocabulary for discussing solutions — not rules to impose. If applying a pattern makes your code more complex and harder to understand, don't use it. Simplicity beats cleverness.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
refactoring.guru	Website	Free	★★★★★	Best visual pattern explanations with interactive examples — start here
Head First Design Patterns	Book	Paid	★★★★★	Most beginner-friendly treatment of patterns — highly recommended
Gang of Four (GoF)	Book	Paid	★★★★■	The original — dense but authoritative, use as reference after Head First
ArjanCodes YouTube	Video	Free	★★★★★	Excellent Python-specific OOP and patterns — clean, modern code

CS203	Databases & SQL	Free
Duration	6 weeks	Intermediate
Est. Hours	~80 focused hours	
Prereqs	CS101	

Almost every system you build stores data. SQL is 50 years old and is used in every company on the planet. Understanding databases deeply — both relational and NoSQL — is what makes full-stack engineers truly effective.

Curriculum Breakdown

SQL Fundamentals

- SELECT, FROM, WHERE, ORDER BY, LIMIT, OFFSET
- JOINS: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF — understand when each returns what
- GROUP BY, HAVING, aggregate functions: COUNT, SUM, AVG, MAX, MIN
- Subqueries and correlated subqueries — when and why
- CTEs (Common Table Expressions) — cleaner than nested subqueries, recursive CTEs
- Window functions: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, PARTITION BY, ROWS BETWEEN
- INSERT, UPDATE, DELETE, UPSERT (INSERT ON CONFLICT DO UPDATE)
- Transactions: BEGIN, COMMIT, ROLLBACK, SAVEPOINT — atomicity in practice
- ACID: Atomicity, Consistency, Isolation (levels), Durability — what each means

Database Design

- Entity-Relationship (ER) diagrams — draw before writing a single line of code
- Primary keys, foreign keys, unique constraints, check constraints, not null
- Normalization: 1NF (atomic values) → 2NF (no partial dependency) → 3NF (no transitive dependency) → BCNF
- Denormalization: when to deliberately break rules for performance
- Many-to-many relationships with junction tables — the most common design pattern
- Indexing: B-tree (range queries), Hash (equality), when to add, when NOT to
- EXPLAIN / EXPLAIN ANALYZE: read query plans, spot sequential scans, understand costs
- Views, materialized views, stored procedures (know they exist and when useful)

NoSQL Databases

- Why NoSQL: document, key-value, column-family, graph — when SQL is the wrong tool
- MongoDB: documents, collections, BSON types, CRUD, aggregation pipeline, \$lookup
- Indexes in MongoDB: compound, text, geospatial
- Redis: strings, hashes, lists, sets, sorted sets — and when to use each
- Redis as cache: cache-aside pattern, TTL, cache invalidation — the hard problem
- Redis pub/sub, Redis streams
- CAP theorem: you can only have 2 of 3 — know the real-world tradeoffs
- BASE vs ACID — eventual consistency and what it means in practice

ORM & Database Integration

- SQLAlchemy Core and ORM: models, relationships (one-to-many, many-to-many), sessions
- Alembic: database migrations — never edit the schema manually in production
- Connection pooling: why single connections fail under load, pool_size, max_overflow
- N+1 query problem: how to detect it, how to fix with eager loading or joins
- Async database: asyncpg, SQLAlchemy async — essential for FastAPI
- Database GUI tools: DBeaver (free, excellent), TablePlus (paid, beautiful)

Common Mistakes — 90% of self-learners get these wrong

- Not drawing an ER diagram before writing code — always design first
- Adding indexes to every column — indexes slow down writes, only add where needed
- Using ORM for complex queries — sometimes raw SQL is cleaner and faster
- Not understanding transactions — missing BEGIN/COMMIT causes data corruption bugs
- Treating NoSQL as 'easier SQL' — they solve different problems

Practice Projects

Project	Description	Lvl	Hireable Version Has...
E-Commerce Schema	Design full schema: users, products, orders, reviews, inventory	Int	3NF normalized, ER diagram first, indexes on all FKs, sample queries documented
SQL 30 Challenge	Write 30 SQL queries of increasing complexity	Int	Must include: window functions, CTEs, correlated subqueries, recursive CTE

Dual-DB App	Python app using both PostgreSQL and MongoDB	Int	SQLAlchemy for relational data, PyMongo for document data, Redis for caching
-------------	--	-----	--

Self-Check — Can you answer these?

- Can you write a query with 3 JOINS and a window function without looking it up?
- Can you explain the difference between 2NF and 3NF with a concrete example?
- Can you describe when you would choose MongoDB over PostgreSQL and why?
- Can you write an Alembic migration that adds a column and backfills data?
- Can you read an EXPLAIN ANALYZE output and identify the performance bottleneck?

When Stuck

If a query is slow: run EXPLAIN ANALYZE before changing anything. If you see 'Seq Scan' on a large table, you need an index. Add it and run again. Measure before optimizing.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
sqlzoo.net	Interactive	Free	★★★★★	Best interactive SQL practice — complete every tutorial and quiz
CMU 15-445 (Andy Pavlo)	Course	Free	★★★★★	World-class database course, free on YouTube — watch every lecture
PostgreSQL Official Docs	Docs	Free	★★★★★	Best SQL reference available — bookmark and use constantly
DDIA (Kleppmann)	Book	Paid	★★★★★	Designing Data-Intensive Applications — most important tech book, read in Phase 3

CS204	Computer Networks	Free
Duration	5 weeks	Intermediate
Est. Hours	~65 focused hours	
Prereqs	CS103	

The internet is the backbone of every application you will ever build. Understanding how data moves from a browser to a server and back will save you hours of debugging connection issues, CORS errors, and performance problems.

Curriculum Breakdown

Network Fundamentals

- OSI model: 7 layers — know what each handles and which protocols live there
- TCP/IP model: 4 layers in practice (Link, Internet, Transport, Application)
- IP addressing: IPv4/IPv6, subnets, CIDR notation, private vs public ranges
- DNS: how domain names resolve — trace with dig or nslookup step by step
- DHCP: how your device gets an IP address automatically
- NAT: how your router shares one public IP across many devices
- ARP: how IP addresses map to MAC addresses on local networks
- Routing: how packets find their path across the internet

Transport Layer

- TCP: three-way handshake (SYN, SYN-ACK, ACK) — draw it from memory
- TCP: reliability (sequence numbers, acknowledgments, retransmission)

- TCP: flow control (receive window) and congestion control (slow start, AIMD)
- TCP connection teardown: FIN, FIN-ACK, ACK — four-way handshake
- UDP: stateless, connectionless, no guarantee — use cases: DNS, video, gaming
- Ports: 0-1023 well-known, 1024-49151 registered, 49152-65535 ephemeral
- Sockets: the API that programs use to communicate over a network

Application Layer

- HTTP/1.1: request/response cycle, methods (GET POST PUT DELETE PATCH HEAD OPTIONS), status codes, headers
- HTTP/2: multiplexing (multiple requests over one TCP connection), header compression, server push
- HTTP/3 and QUIC: UDP-based, reduces latency, built-in encryption
- HTTPS and TLS: TLS handshake, certificates, certificate authorities, certificate pinning
- WebSockets: bidirectional persistent connection — how real-time apps work
- REST principles: stateless, uniform interface, resource-based URIs
- gRPC: Protocol Buffers, strongly typed, faster than REST — when to use it
- SMTP, FTP, SSH, DNS (application layer): know what each does

Security Fundamentals

- Symmetric encryption: AES — same key for encrypt/decrypt, fast, used for bulk data
- Asymmetric encryption: RSA — public/private key pair, used for key exchange
- TLS certificate chain: root CA, intermediate CA, leaf certificate
- Hashing: SHA-256, bcrypt for passwords — never store plaintext or MD5
- Common attacks: MITM, DDoS, DNS spoofing, replay attacks, SSL stripping
- CORS: why browsers enforce it, preflight requests, how to configure correctly
- JWT: header.payload.signature, where to store (httpOnly cookie vs localStorage)
- OAuth 2.0 and OIDC: authorization vs authentication — the difference matters

Common Mistakes — 90% of self-learners get these wrong

Memorizing OSI layers without understanding what they actually do

Not using Wireshark to see real traffic — theory without observation is weak

Confusing authentication (who are you?) with authorization (what can you do?)

Not understanding HTTPS — many developers don't know what TLS actually does

Practice Projects

Project	Description	Lvl	Hireable Version Has...
HTTP Server	Build minimal HTTP/1.1 server from raw Python sockets	Adv	Handles GET/POST, serves static files, parses headers, returns correct status codes
TCP Chat Server	Multi-client chat using sockets and threading	Int	Handles disconnections gracefully, broadcasts to all clients, rooms support
Packet Capture Analysis	Wireshark: capture and document real traffic	Int	Document: HTTP request, DNS resolution, TCP handshake, TLS handshake with screenshots

Self-Check — Can you answer these?

- ☐ Can you draw the TCP three-way handshake from memory?
- ☐ Can you describe what happens at each OSI layer when you visit google.com?
- ☐ Can you explain why HTTPS prevents MITM attacks — what specifically does TLS prevent?
- ☐ Can you explain CORS: what it is, why browsers enforce it, and how to fix a CORS error?
- ☐ Can you explain the difference between authentication and authorization?

When Stuck

If network concepts feel abstract: open Wireshark and browse the internet. Watch the TCP handshake happen live. Watch DNS resolve in real time. Theory becomes obvious when you see it.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Stanford CS144	Course	Free	★★★★★	Best university networking course — free on YouTube, assignments included
Wireshark	Tool	Free	★★★★★	Install and capture your own traffic — hands-on is essential
Julia Evans zines	Zines	Free	★★★★★	Networking for programmers — approachable, accurate, beautifully done
Kurose & Ross	Book	Paid	★★★★★	Computer Networking: A Top-Down Approach — standard university textbook

CS205	Application Security	Free
Duration	3 weeks	Intermediate
Est. Hours	~45 focused hours	
Prereqs	CS203, CS204	

Security is not a feature you add at the end — it is a mindset you build into every line of code. Filipino fintech companies (GCash, Paymaya, UnionDigital) explicitly test security knowledge. This subject makes you dangerous in the best way.

Curriculum Breakdown

OWASP Top 10 (The Must-Knows)

- A01 Broken Access Control: insecure direct object reference, missing authorization checks
- A02 Cryptographic Failures: weak algorithms, unencrypted sensitive data, hardcoded keys
- A03 Injection: SQL injection — understand and prevent with parameterized queries
- A03 XSS (Cross-Site Scripting): reflected, stored, DOM-based — escape output always
- A04 Insecure Design: threat modeling, fail securely by default, principle of least privilege
- A05 Security Misconfiguration: default credentials, verbose errors, open S3 buckets
- A07 Authentication Failures: brute force, credential stuffing, session fixation
- A08 Software Supply Chain: vulnerable dependencies, lock files, dependency scanning
- A09 Logging Failures: no audit trail, logging sensitive data, insufficient monitoring
- A10 SSRF: making the server request internal resources — cloud metadata endpoint attacks

Secure Coding Practices

- Never trust user input — validate and sanitize everything on the server side
- Parameterized queries: NEVER use string formatting for SQL — ever
- Password storage: bcrypt with cost factor 12+, never MD5 or SHA1 for passwords
- Secrets management: environment variables, .env files, never commit to git
- Rate limiting: per-IP, per-user, per-endpoint — prevent brute force
- CSRF protection: SameSite cookies, CSRF tokens for state-changing requests
- Secure headers: HSTS, CSP, X-Frame-Options, X-Content-Type-Options
- Input validation: allowlist not denylist, validate type/length/format/range

Security in Practice

- Dependency scanning: pip-audit, Safety, Snyk — run in CI pipeline
- HTTPS everywhere: redirect HTTP to HTTPS, HSTS preloading
- JWT security: short expiry, httpOnly cookies, refresh token rotation, token revocation
- OAuth 2.0 security: state parameter (CSRF prevention), PKCE for public clients

- Error handling: never expose stack traces in production responses
- Logging: log security events (failed logins, permission denials) without logging PII
- Penetration testing basics: OWASP ZAP, Burp Suite Community — scan your own apps
- Bug bounty mindset: think like an attacker to defend like a defender

Common Mistakes — 90% of self-learners get these wrong

Thinking security is someone else's job — every engineer writes security-relevant code

Storing secrets in code or environment variables tracked by git

Using string formatting to build SQL queries — this is still the #1 SQL injection cause

Logging too much (PII, passwords) or too little (no audit trail)

Not scanning dependencies — the next Log4Shell could be in your requirements.txt

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Vulnerable App Audit	Find and fix all OWASP Top 10 issues in a deliberately vulnerable app	Int	Use DVWA or WebGoat, document each vulnerability found and the fix applied
Secure API	Build a FastAPI endpoint following all security best practices	Int	Rate limiting, CSRF, parameterized queries, secure headers, dependency scan, secrets in env

Self-Check — Can you answer these?

- ☐ Can you explain SQL injection and write a parameterized query that prevents it?
- ☐ Can you explain the difference between XSS and CSRF?
- ☐ Can you explain why MD5 is unsafe for password storage?
- ☐ Can you describe 5 secure HTTP headers and what each one does?
- ☐ Can you explain what SSRF is and give a real attack example?

When Stuck

If you're confused about whether something is a vulnerability: ask 'what happens if a malicious user controls this input?' If the answer is anything bad, it's a vulnerability. Always assume the attacker knows your code.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
OWASP Top 10	Docs	Free	★★★★★	owasp.org — read the official descriptions and prevention guides
PortSwigger Web Academy	Interactive	Free	★★★★★	portswigger.net/web-security — hands-on labs, best free security training
DVWA	Tool	Free	★★★★★	Damn Vulnerable Web App — practice finding vulnerabilities in a safe environment
Hacking: The Art of Exploitation	Book	Paid	★★★★★	Low-level security — read after OWASP basics are solid

Phase Capstone

Build a secure, production-quality REST API for a personal finance tracker: users, transactions, categories, monthly reports. Requirements: parameterized queries only, bcrypt passwords, JWT with refresh tokens, rate limiting, CSRF protection, secure headers, input validation, dependency scan in CI, EXPLAIN ANALYZE on every query, Redis caching, full OWASP audit documented. Deploy to a free cloud service. This is portfolio piece #2.

PHASE 3

~580 hrs

Full-Stack Engineering

Build real things. Ship real products. This is your job-readiness phase.

Est. Duration	5–7 months	Est. Total Hours	~580 hours
---------------	------------	------------------	------------

Must Complete (MVP)	Nice-to-Have (If Time Allows)
• CS301 Backend (FastAPI + REST + Security)	• CS303 System Design (vocabulary now, mastery on the job)
• CS302 Frontend (JS + React + TypeScript)	• Advanced React patterns
• CS305 Cloud Fundamentals (AWS/GCP)	
• CS304 CI/CD basics	

Interview Habit

From Phase 3: do one full system design mock per month. Set a 45-minute timer. Draw on paper. No notes. Review against ByteByteGo afterward. Also: start applying to junior roles — you don't need to wait for Phase 5.

CS301	Backend Development	Free
Duration	10 weeks	Intermediate-Hard
Est. Hours	~150 focused hours	
Prereqs	CS101, CS203, CS204, CS205	

The backend is the engine of every application. It processes logic, manages data, enforces security, and powers every feature users see. Python is your language. FastAPI is your framework. Security from CS205 is mandatory here — every endpoint you build applies those lessons.

Curriculum Breakdown

Advanced Python for Backend

- Decorators: write your own — @cache, @retry, @require_auth
- Generators and iterators: lazy evaluation, memory-efficient data processing
- async/await: event loop, coroutines, asyncio.gather, asyncio.TaskGroup
- Type hints: complete typing with mypy, no implicit Any
- Pydantic v2: validators, field aliases, model serialization, computed fields
- Context managers: __enter__ / __exit__, asynccontextmanager
- Concurrency: threading vs multiprocessing vs asyncio — when each applies
- functools: lru_cache, partial, reduce — functional patterns

FastAPI Deep Dive

- Routes: GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS
- Path params, query params, request body, form data, file uploads
- Pydantic schemas: request validation, response models, nested models
- Dependency injection: shared DB sessions, current user, feature flags
- Authentication: JWT with python-jose, refresh token rotation, token blacklisting

- OAuth2 password flow and Bearer token — complete implementation
- Middleware: CORS (configured correctly), rate limiting (slowapi), request logging
- Background tasks: fire-and-forget, Celery for heavy async work
- WebSockets in FastAPI: real-time connections, connection manager
- Auto docs: Swagger UI customization, ReDoc, OpenAPI schema export

RESTful API Design Excellence

- REST constraints: stateless, uniform interface, HATEOAS overview
- Resource naming: plural nouns, nested resources, no verbs in URIs
- HTTP methods used semantically: PUT (replace) vs PATCH (partial update)
- Status codes: use the right one — 201 not 200 for creation, 204 for delete
- Pagination: cursor-based (preferred) vs offset, include total count, next/prev links
- Filtering: query params, complex filters with operators (gt, lt, contains)
- Sorting: multiple fields, direction, default sort documented
- Versioning: URI versioning (/v1/) vs header versioning — tradeoffs
- Error responses: RFC 7807 Problem Details format — consistent error structure
- API documentation: write it as if you'll never be available to explain it

Database Integration & Performance

- SQLAlchemy 2.0: declarative models, relationships, lazy vs eager loading
- Alembic: autogenerate migrations, upgrade/downgrade, data migrations
- Async SQLAlchemy with asyncpg: connection pool configuration
- N+1 query detection: SQLAlchemy query logging, detect and fix with joinedload/selectinload
- Database seeding and fixtures for testing
- Repository pattern: abstract database access from business logic
- Optimistic locking: version columns to prevent lost updates

Testing, Quality & Observability

- pytest: test functions, fixtures, conftest.py, parametrize
- FastAPI TestClient for integration tests, async test client
- Mocking: unittest.mock, pytest-mock, monkeypatching
- Test-driven development: red-green-refactor — write the test first
- Code coverage: pytest-cov, 80% minimum, 100% for critical paths
- Structured logging: JSON logs, correlation IDs, log levels
- Error monitoring: Sentry integration — every exception tracked in production
- Health check endpoints: /health, /ready — required for cloud deployment

Docker & Deployment

- Docker: multi-stage Dockerfile (build stage + slim runtime stage)
- docker-compose: app + PostgreSQL + Redis + pgAdmin
- .env files, python-dotenv, secret rotation strategy
- NGINX: reverse proxy config, SSL termination, gzip, rate limiting at proxy level
- Gunicorn + Uvicorn workers: how many workers, tuning for your machine
- Deploying to Railway/Render (free tier) and understanding the config
- GitHub Actions: test on PR, lint on PR, deploy on merge to main

Common Mistakes — 90% of self-learners get these wrong

- Not using async properly — mixing sync and async code causes subtle performance bugs
- Returning 200 for everything including errors — use the right status codes
- Not testing error paths — most bugs live in error handling, not the happy path
- Storing secrets in code — even in 'private' repos. Use environment variables always
- Not writing the OpenAPI spec — document your API as you build it, not after

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Blog API	Full REST API: users, posts, comments, likes, tags, auth	Adv	JWT + refresh tokens, cursor pagination, Redis cache, full test suite, deployed with CI/CD
Task Manager API	Team tasks: workspaces, members, roles, assignments, notifications	Adv	RBAC, file attachments to S3, WebSocket notifications, Celery background jobs, Sentry
Live Production Deployment	Deploy Blog API with full production config	Adv	HTTPS, env vars in cloud, GitHub Actions CI/CD, health checks, Sentry, structured logging

Self-Check — Can you answer these?

- ☐ Can you build a JWT + refresh token auth system from scratch in under 2 hours?
- ☐ Can you explain the difference between PUT and PATCH semantically?
- ☐ Can you write a complete pytest suite for a FastAPI route including error cases?
- ☐ Can you write a multi-stage Dockerfile that produces a slim production image?
- ☐ Can you set up GitHub Actions that lints, tests, builds Docker, and deploys on merge?

When Stuck

If your API is slow: profile with py-spy or cProfile before guessing. 80% of slowness is an N+1 query to the database. Use SQLAlchemy's echo=True to log all SQL and spot the issue.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
FastAPI Docs	Docs	Free	★★★★★	fastapi.tiangolo.com — best framework documentation ever written, read everything
ArjanCodes YouTube	Video	Free	★★★★★	Python architecture and clean backend code — excellent channel
TestDriven.io FastAPI	Course	Paid	★★★★★	FastAPI TDD — best paid course for backend testing in Python
Real Python	Articles	Free	★★★★★	realpython.com — deep-dive Python backend articles, well written

CS302	Frontend Development	Free
Duration	10 weeks	Intermediate
Est. Hours	~140 focused hours	
Prereqs	CS101, CS105	

Frontend engineering is not graphic design — it is building interfaces that are correct, fast, and accessible. You already know TypeScript from CS105. Now you apply it to React. Use Tailwind CSS for styling so you focus on functionality, not design decisions.

Curriculum Breakdown

HTML5 & CSS Mastery

- HTML5 semantics: header, main, section, article, nav, footer, aside, figure
- Forms: inputs, labels, fieldsets, validation attributes, accessibility (ARIA)
- CSS box model: content, padding, border, margin — draw it, know it cold
- Flexbox: container properties, item properties, alignment, gap — use it for 1D layouts
- CSS Grid: template columns/rows, areas, auto-placement, subgrid — use it for 2D layouts

- Responsive design: mobile-first, breakpoints, fluid typography, container queries
- CSS custom properties: `--color-primary`, cascading, theming
- CSS animations and transitions: transform, opacity, keyframes, will-change
- CSS specificity and the cascade: understand before using `!important`

JavaScript Deep Dive

- `var/let/const`: differences, hoisting, temporal dead zone, block scope
- Arrow functions vs regular functions: this binding — the source of 1000 bugs
- Closures: functions that remember their outer scope — iterators, private state, memoization
- Prototypal inheritance: prototype chain, `Object.create`, class as syntax sugar
- Event loop: call stack, microtask queue, macrotask queue — why `setTimeout(fn, 0)` is not instant
- Promises: `.then()`, `.catch()`, `.finally()`, `Promise.all()`, `Promise.allSettled()`, `Promise.race()`
- `async/await`: syntactic sugar over promises, error handling with `try/catch`
- Fetch API: GET, POST, headers, body, error handling, `AbortController`
- DOM API: `querySelector`, `addEventListener`, event delegation, `MutationObserver`
- ES2020+: optional chaining `?.`, nullish coalescing `??`, logical assignment, `structuredClone`
- Modules: `import/export`, `dynamic import()`, tree-shaking implications

TypeScript in React

- Typing React components: FC, explicit return types
- Typing props: required, optional, union types, callback props
- Typing `useState`: `useState(null)`
- Typing `useRef`: `useRef(null)`
- Typing events: `React.ChangeEvent`, `React.MouseEvent`, `React.FormEvent`
- Generic components: `List`, `Select`
- Typing context with `createContext` and proper defaults
- Discriminated unions for component variants

React Architecture

- JSX: how Babel transforms it, JSX is just function calls
- Functional components: why class components are legacy
- Props: data flows down — unidirectional data flow
- `useState`: immutability, state batching in React 18, functional updates
- `useEffect`: effects, dependency array, cleanup — the three variants and when to use each
- `useContext`: avoid prop drilling — when context is and isn't appropriate
- `useReducer`: complex state, dispatch pattern, integrating with context
- Custom hooks: extract and reuse stateful logic — the superpower of hooks
- React Router v6: `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `useParams`, `useNavigate`, `useLocation`
- Performance: `React.memo`, `useMemo`, `useCallback` — understand when NOT to use them (premature optimization)
- Error boundaries: class component, `react-error-boundary` library, fallback UI
- `Suspense` and lazy: code splitting, dynamic imports for route-level chunks

Frontend Tooling & Architecture

- Vite: fast dev server with HMR, build with Rollup, environment variables
- `npm/yarn/pnpm`: `package.json`, `package-lock.json`, peer dependencies
- Tailwind CSS: utility-first, responsive variants, dark mode, arbitrary values
- Zustand: simple global state — when context isn't enough but Redux is overkill
- TanStack Query (React Query): server state, caching, background refetch, optimistic updates
- Axios vs Fetch: interceptors, request/response transformation, cancellation
- Zod: runtime schema validation — pair with TypeScript for end-to-end type safety
- Testing: Vitest (fast, Vite-native), React Testing Library (user-centric tests)
- Storybook: component documentation and visual testing — used in real teams
- Accessibility: WCAG AA, keyboard navigation, screen reader testing, axe DevTools

Common Mistakes — 90% of self-learners get these wrong

Not using TypeScript strictly — any and `@ts-ignore` defeat the purpose

Putting all state in Context — most state should be local, not global

Not testing from the user's perspective — test what the user sees, not implementation details

Ignoring accessibility — broken keyboard navigation fails interviews at companies that care

Over-using useEffect — most side effects belong elsewhere (event handlers, React Query)

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Portfolio Site	Custom responsive site, zero templates	Int	Mobile-first, dark mode, animations, WCAG AA accessible, deployed on Vercel, Lighthouse 90+
Weather Dashboard	OpenWeatherMap API, TypeScript, React Query	Int	5-day forecast, charts with Recharts, geolocation, skeleton loading, error states
Full-Stack Task App	React + TypeScript consuming CS301 Task API	Adv	Auth flow, optimistic updates, real-time via WebSocket, PWA, offline support
Real-Time Chat UI	WebSocket chat with full TypeScript	Adv	Typing indicators, read receipts, message history, rooms, emoji reactions

Self-Check — Can you answer these?

- ☐ Can you explain the JavaScript event loop with a code example that surprises most developers?
- ☐ Can you build a fully typed React form with validation using Zod and React Hook Form?
- ☐ Can you explain all three forms of useEffect dependency array and when each is appropriate?
- ☐ Can you implement an optimistic update in TanStack Query from scratch?
- ☐ Can you score 90+ on Lighthouse for your portfolio site?

When Stuck

If React re-renders are confusing: install React DevTools, enable 'Highlight updates when components render'. You'll see exactly what re-renders and why. Add React.memo only after you see a problem.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
javascript.info	Website	Free	★★★★★	Most thorough JS resource ever written — read every chapter
react.dev	Docs	Free	★★★★★	Official React docs, completely rewritten, excellent — primary reference
Total TypeScript (React)	Course	Paid	★★★★★	Best typed React patterns — worth the investment
The Odin Project	Course	Free	★★★★★	Project-based full curriculum for HTML/CSS/JS fundamentals

CS303

System Design

Free/Paid

Duration	8 weeks	Hard
Est. Hours	~90 focused hours	
Prereqs	CS203, CS204, CS301	

System design is how senior engineers think. It's about designing systems that serve millions of users without falling over. You won't master this in 8 weeks — but you must build the vocabulary now and deepen

it on the job through every system you touch.

Curriculum Breakdown

Scalability Foundations

- Vertical vs horizontal scaling: when each applies, limits of each
- Load balancers: round-robin, least connections, IP hashing, sticky sessions
- Caching strategies: cache-aside (lazy), write-through, write-behind, read-through
- Cache eviction policies: LRU, LFU, FIFO — which to use when
- CDNs: push vs pull, edge caching, cache invalidation, geographic distribution
- Database scaling: read replicas (lag), sharding (horizontal partitioning), partitioning strategies
- Consistent hashing: how distributed systems map keys to nodes without remapping everything

Core Infrastructure Components

- Message queues: async processing, decoupling producers from consumers, at-least-once delivery
- Kafka: topics, partitions, consumer groups, offset management, exactly-once semantics
- RabbitMQ vs Kafka vs SQS: when each is the right tool
- Microservices vs monolith: the real tradeoffs, not the hype — start monolith, extract services
- API gateway: single entry point, routing, rate limiting, auth, request transformation
- Service discovery: Consul, Kubernetes service discovery — how services find each other
- Circuit breaker pattern: prevent cascade failures, Hystrix, resilience4j concepts

Design Case Studies

- URL Shortener: hash function, collision handling, custom aliases, analytics, TTL
- Social Feed (Twitter): fanout on write vs fanout on read — when each wins
- Video Streaming (YouTube): chunking, transcoding pipeline, adaptive bitrate, CDN strategy
- Ride-Sharing (Grab/Uber): geospatial indexing (geohash, quadtree), driver matching, surge pricing
- Chat Application (Slack): WebSockets, message ordering, presence, search at scale
- Notification System: fan-out, push vs pull, SMS/email/push at scale, delivery receipts
- Distributed File Storage (Google Drive): chunking, deduplication, sync protocol, conflict resolution
- Search Engine: inverted index, TF-IDF, ranking signals, query parsing

Reliability & Operations

- CAP theorem: you can only have 2 — know which 2 each major system chooses and why
- PACELC: extends CAP — latency vs consistency tradeoff when no partition
- Fault tolerance: replication factor, failover, circuit breaker, bulkhead pattern
- Monitoring: the four golden signals (latency, traffic, errors, saturation)
- SLA/SLO/SLI: definitions, how to set realistic targets, error budgets
- Distributed tracing: correlation IDs, Jaeger, OpenTelemetry, trace sampling
- Chaos engineering: deliberately break things to find weaknesses before users do

Common Mistakes — 90% of self-learners get these wrong

Starting with solutions before clarifying requirements — always clarify scale first

Not making explicit tradeoffs — 'it depends' without explanation is not an answer

Over-engineering for a startup's scale — design for 10x your current load, not 1000x

Not knowing the numbers: 1ms RAM access, 1ms network, 10ms disk — know these cold

Designing everything as microservices from day one — start with a monolith

Practice Projects

Project	Description	Lvl	Hireable Version Has...
URL Shortener	End-to-end with Redis caching + analytics	Adv	Custom aliases, click analytics dashboard, expiry, rate limiting, deployed, load tested
Design Documents	Architecture docs for 5 case studies	Adv	Each: requirements, capacity estimates, component diagram, data model, API design, tradeoffs

Self-Check — Can you answer these?

- ☐ Can you design a URL shortener end-to-end in 45 minutes including tradeoffs?
- ☐ Can you explain CAP theorem with a concrete example of each combination?
- ☐ Can you describe 3 different strategies to scale a read-heavy PostgreSQL database?
- ☐ Can you explain what a message queue solves that a synchronous API call cannot?
- ☐ Can you describe the fanout problem in social feeds and both solutions?

When Stuck

In a system design interview: always start by clarifying requirements and estimating scale. Jumping to solutions without scope is the #1 mistake senior engineers see. 'Let me clarify first' always impresses.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
System Design Primer	GitHub	Free	★★★★★	github.com/donnmartin — largest free system design resource
ByteByteGo (Alex Xu)	YouTube/Book	Free/Paid	★★★★★	Best visual explanations — watch free YouTube, buy the book if you can
DDIA (Kleppmann)	Book	Paid	★★★★★	Designing Data-Intensive Applications — the most important tech book
Grokking SD Interview	Course	Paid	★★★★★	educative.io — structured interview format, good if budget allows

CS304	Software Engineering Practices & DevOps	Free
Duration	4 weeks	Intermediate
Est. Hours	~50 focused hours	
Prereqs	CS202, CS301	

Real-world software is a team sport. This is how code goes from an idea in your head to a feature running in production without breaking things. Learn these practices now so your first job doesn't shock you.

Curriculum Breakdown

Agile & Team Workflow

- Agile manifesto: what it actually says vs how it's misused
- Scrum: sprints (2 weeks), daily standups (15 min max), sprint review, retrospective
- Kanban: visualize work, WIP limits, cycle time — good for ops and maintenance work
- Git workflow: Gitflow vs trunk-based development — when each works
- PR discipline: small PRs (< 400 lines), good descriptions, self-review before requesting review
- Code review culture: be kind, be specific, explain the why, distinguish blocker vs nit
- Conventional commits: feat, fix, docs, chore, refactor, test, perf, ci

CI/CD Pipeline Mastery

- CI: run tests automatically on every push — non-negotiable
- CD: deploy automatically on merge to main — not just possible but expected
- GitHub Actions: YAML syntax, jobs, steps, actions marketplace, secrets management
- Writing efficient workflows: caching dependencies, parallel jobs, matrix builds
- Docker in pipelines: build image, push to registry (Docker Hub, GHCR), tag strategy
- Environment management: dev, staging, production — promote through environments

- Feature flags: deploy code without activating features — LaunchDarkly, custom flags
- Rollback strategy: how to revert a bad deployment safely

Code Quality & Professionalism

- Code review: what to look for — correctness, security, performance, readability, tests
- Technical debt: categorize it (intentional vs accidental), track it, pay it down regularly
- Refactoring safely: rename, extract function, extract class, inline, move — with tests
- Documentation: docstrings (Google/NumPy style), README (what, why, how, examples)
- ADR (Architecture Decision Records): document why you made a decision, not just what
- Structured logging: JSON format, correlation IDs, log levels, what to log and what not to
- Error monitoring: Sentry — every unhandled exception tracked, alerted, assigned

Engineering Soft Skills

- Writing clear bug reports: reproduction steps, expected vs actual, environment, logs
- Asking for help effectively: show what you tried, share the error, share context
- Technical communication: explain decisions without jargon to non-technical stakeholders
- Estimating work: range estimates, what you know vs what you need to learn
- Giving and receiving critical feedback: direct, kind, specific, actionable
- Meeting discipline: come prepared, take notes, confirm decisions, follow up in writing

Common Mistakes — 90% of self-learners get these wrong

Writing PRs that are 2000 lines — nobody reviews them properly, they just approve

Skipping the staging environment — deploying dev directly to prod causes incidents

Treating CI as optional — CI is your safety net, never disable it

Not writing ADRs — you will forget why you made architectural decisions without them

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Full CI/CD Pipeline	GitHub Actions for Blog API: test, lint, build, deploy	Int	Tests on every PR, build Docker on merge, deploy to cloud, Sentry alerts on failure
Open Source Contribution	Real PR merged to an open source project	Int	Fix a real bug or add a real feature — document the full process in your portfolio

Self-Check — Can you answer these?

- ☐ Can you write a complete GitHub Actions workflow from scratch including matrix builds?
- ☐ Can you explain the difference between CI and CD and give examples of each?
- ☐ Can you write a constructive, kind, and specific code review comment on a junior's PR?
- ☐ Can you write an Architecture Decision Record for a technology choice you made?

When Stuck

If a CI pipeline fails: read the log from the top, not the bottom. Every failure has a root cause clearly stated. Jumping to the bottom and guessing wastes 30 minutes. Read top to bottom, once.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
GitHub Actions Docs	Docs	Free	★★★★★	Official docs — comprehensive, well-structured, search when you need something
Google Engineering Practices	Guide	Free	★★★★★	google.github.io/eng-practices — how Google does code review, read fully

The Pragmatic Programmer	Book	Paid	★★★★★	Hunt & Thomas — essential reading for every professional engineer
Accelerate (Forsgren)	Book	Paid	★★★★★	Science of DevOps — evidence-based engineering practices

CS305	Cloud Fundamentals (AWS & GCP)	Free/Paid
Duration	4 weeks	Intermediate
Est. Hours	~70 focused hours	
Prereqs	CS301, CS304	

Cloud is not optional in 2025 — it is where every application runs. AWS and GCP are listed in the majority of PH tech job postings for mid-level and above. Understanding cloud fundamentals makes you immediately more productive on day one of any job.

Curriculum Breakdown

AWS Core Services

- IAM: users, groups, roles, policies, principle of least privilege — security starts here
- EC2: instances, AMIs, security groups, key pairs, user data scripts, instance types
- VPC: subnets (public vs private), route tables, internet gateways, NAT gateways
- S3: buckets, objects, storage classes, versioning, lifecycle policies, pre-signed URLs
- RDS: managed PostgreSQL/MySQL, parameter groups, snapshots, read replicas, Multi-AZ
- ElastiCache: managed Redis/Memcached — cache your database queries
- Lambda: serverless functions, triggers (API Gateway, S3, SQS), cold starts, limits
- API Gateway: REST vs HTTP APIs, rate limiting, usage plans, Lambda integration
- CloudWatch: logs, metrics, alarms, dashboards — your observability layer
- Route 53: DNS management, health checks, routing policies

GCP Core Services

- IAM: service accounts, roles, bindings — different from AWS IAM but same concepts
- Compute Engine: VMs, machine types, preemptible instances — equivalent to EC2
- Cloud Storage: buckets, ACLs, signed URLs — equivalent to S3
- Cloud SQL: managed PostgreSQL/MySQL — equivalent to RDS
- Cloud Run: serverless containers — easier than Lambda for web APIs, highly recommended
- BigQuery: serverless data warehouse — SQL on petabyte-scale data, pay per query
- Cloud Build: CI/CD pipelines — GitHub Actions alternative in GCP ecosystem
- Vertex AI: managed ML platform — where CS403 models go in production
- Cloud Functions: event-driven serverless — equivalent to Lambda

Cloud Architecture Patterns

- Three-tier architecture: load balancer > app servers > database — the classic pattern
- Serverless architecture: API Gateway + Lambda/Cloud Run + managed DB — low ops overhead
- Container architecture: Kubernetes overview — pods, services, deployments, ingress
- Infrastructure as Code: Terraform basics — define infrastructure in code, version it, review it
- Cost management: spot/preemptible instances, right-sizing, reserved instances, billing alerts
- Security: security groups, NACLs, VPC flow logs, CloudTrail, GuardDuty — defense in depth
- High availability: multi-AZ deployment, health checks, auto-scaling groups

Cloud in Practice

- Deploying a FastAPI app to Cloud Run: Dockerfile, Cloud Build trigger, environment variables
- Setting up a PostgreSQL instance on Cloud SQL or RDS with connection pooling
- Storing user-uploaded files in S3/Cloud Storage with pre-signed URLs
- Setting up CloudWatch/Cloud Logging dashboards for your applications
- Cost estimation: AWS Pricing Calculator, GCP Pricing Calculator — always estimate first

- AWS/GCP Free Tier: what's actually free, what has usage limits, how to avoid surprise bills
- Certification roadmap: AWS Solutions Architect Associate or GCP Associate Cloud Engineer

Common Mistakes — 90% of self-learners get these wrong

- Using the root AWS account for everything — always create IAM users with least privilege
- Not setting billing alerts — surprise cloud bills happen to experienced engineers too
- Not understanding the free tier limits — some services are free up to a limit, then expensive
- Treating cloud as just 'someone else's computer' — understanding the primitives matters
- Not using IaC (Terraform) — clicking in the console doesn't scale and isn't reproducible

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Cloud-Deployed API	Deploy CS301 Blog API on Cloud Run or EC2	Int	VPC with private subnet for DB, S3 for files, Redis on ElastiCache, CloudWatch alerts
Serverless Image Processor	Lambda/Cloud Function that resizes uploaded images	Int	Triggered by S3 upload, stores resized versions, no server to manage, pay per invocation

Self-Check — Can you answer these?

- ☐ Can you explain the difference between a security group and a NACL in AWS?
- ☐ Can you deploy a containerized FastAPI app to Cloud Run using Cloud Build?
- ☐ Can you explain what IAM roles are and why they are more secure than IAM users?
- ☐ Can you estimate the monthly cost of running a small web app on AWS?
- ☐ Can you explain the difference between Lambda and Cloud Run and when to use each?

When Stuck

If a cloud deployment fails: check the logs first (CloudWatch Logs / Cloud Logging). 90% of cloud issues are: wrong IAM permissions, misconfigured security groups, or environment variable not set. Check these three things first.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
AWS Free Tier	Platform	Free	★★★★★	aws.amazon.com/free — create an account and build with free tier services
Cloud Run Documentation	Docs	Free	★★★★★	cloud.google.com/run — simplest way to deploy containers, excellent docs
AWS Solutions Architect Study	Course	Free/Paid	★★★★★	Adrian Cantrill or Stephane Maarek — both excellent for SAA-C03 certification
Terraform Getting Started	Docs	Free	★★★★★	developer.hashicorp.com/terraform/tutorials — official, hands-on

Phase Capstone

Build and deploy a complete SaaS application: a team project management tool (like a mini Linear/Jira). Requirements: FastAPI backend with RBAC auth, React + TypeScript frontend, PostgreSQL on Cloud SQL/RDS, Redis caching, file uploads to S3/Cloud Storage, real-time notifications via WebSocket, full CI/CD pipeline, monitoring with CloudWatch/Cloud Logging, Sentry error tracking, and a proper system design document explaining every architectural decision. This is portfolio piece #3 — your most impressive project to date.

PHASE 4

~820 hrs

Machine Learning & AI Engineering

Your ultimate destination. Math meets code. Every phase before this was preparation.

Est. Duration	9–12 months	Est. Total Hours	~820 hours
---------------	-------------	------------------	------------

Must Complete (MVP)	Nice-to-Have (If Time Allows)
• CS401 ML Foundations (all modules)	• CS404 pick ONE specialization track
• CS402 Deep Learning (Modules 1-6)	• CS406 Data Engineering (if targeting data-heavy roles)
• CS403 MLOps + LLM Engineering	
• CS405 Data Visualization	

Interview Habit

From Phase 4: apply to ML roles even before Phase 4 is complete. Companies often hire ML engineers with strong Python/backend skills and train ML on the job. Your CS201-CS305 background already makes you competitive for many ML adjacent roles.

CS401	Machine Learning Foundations	Free
Duration	14 weeks	Hard
Est. Hours	~200 focused hours	
Prereqs	CS102, CS101, CS201	

Every ML algorithm has a mathematical story behind it. Learn the story first — the code follows naturally. The most important skill in ML is not knowing which algorithm to use — it is feature engineering. Spend more time on Module 4 than on anything else.

Curriculum Breakdown

Module 1: ML Fundamentals

- What is ML: supervised, unsupervised, reinforcement, self-supervised — definitions and examples
- The ML pipeline: problem definition > data collection > EDA > preprocessing > feature engineering > modeling > evaluation > deployment > monitoring
- Bias-variance tradeoff: the central tension of all of ML — understand this deeply
- Overfitting: too complex, memorizes training data — detection via train-val gap
- Underfitting: too simple, misses patterns — detection via high training error
- Train/validation/test split: why 3 sets, never touch test until final evaluation
- Cross-validation: k-fold, stratified k-fold (for imbalanced), time-series split
- Evaluation metrics: accuracy (when to use), precision, recall, F1, AUC-ROC, PR curve
- Regression metrics: MSE, RMSE, MAE, MAPE, R-squared — when each is appropriate

Module 2: Supervised Learning Algorithms

- Linear Regression: cost function (MSE), gradient descent, normal equation, assumptions, R^2

- Logistic Regression: sigmoid function, log loss, decision boundary, regularization
- Decision Trees: information gain, Gini impurity, pruning, max_depth, min_samples_leaf
- Random Forest: bagging, out-of-bag error, feature importance — most reliable baseline
- Gradient Boosting: sequential ensembles, XGBoost (know every hyperparameter), LightGBM, CatBoost
- SVM: hyperplane, support vectors, margin, kernel trick (RBF, polynomial) — for small/medium datasets
- K-Nearest Neighbors: distance metrics (Euclidean, Manhattan, cosine), curse of dimensionality
- Naive Bayes: conditional independence assumption, text classification, Laplace smoothing

Module 3: Unsupervised Learning

- K-Means: algorithm, elbow method, silhouette score, limitations (spherical clusters, need K)
- Hierarchical clustering: agglomerative (bottom-up), dendrogram, linkage criteria (Ward)
- DBSCAN: density-based, handles arbitrary shapes, no K needed, handles noise as outliers
- PCA: variance explained, scree plot, when to use it (high-dimensional, multicollinear data)
- t-SNE: visualization only, perplexity parameter, do NOT use for ML features
- UMAP: faster than t-SNE, preserves global structure better, can be used for features
- Association rules: support, confidence, lift, Apriori algorithm, market basket analysis

Module 4: Feature Engineering — The Most Impactful Skill

- EDA: describe(), value_counts(), hist(), pairplot(), heatmap(), identify distributions and outliers
- Missing values: understand WHY data is missing (MCAR, MAR, MNAR) before imputing
- Imputation: mean/median/mode, KNN impute, iterative imputer, flag missingness as a feature
- Categorical encoding: one-hot (low cardinality), label (ordinal), target (high cardinality), binary
- Feature scaling: StandardScaler (normalize), MinMaxScaler (0-1), RobustScaler (outliers present)
- Outlier detection: IQR method, Z-score, Isolation Forest — cap or remove depending on context
- Feature selection: correlation matrix, mutual information, permutation importance, RFE
- Feature creation: datetime features (hour, day, weekend), interaction terms, polynomial features
- Imbalanced datasets: SMOTE (oversample minority), ADASYN, class_weight='balanced', threshold tuning
- Target leakage: the silent killer — never let future information into training features

Module 5: Model Optimization

- Hyperparameter tuning: GridSearchCV (exhaustive), RandomizedSearchCV (efficient), Optuna (Bayesian)
- Optuna: define objective, suggest parameters, pruning, visualization — learn this over GridSearch
- sklearn Pipeline: chain preprocessing + model — prevents data leakage in cross-validation
- Regularization: L1 (Lasso, feature selection), L2 (Ridge, shrinkage), Elastic Net (both)
- Calibration: Platt scaling, isotonic regression — when predicted probabilities must be accurate
- Learning curves: diagnose whether more data or a better model is the right next step
- Ensemble methods: voting (soft/hard), stacking (meta-learner), blending — how Kaggle winners win

Common Mistakes — 90% of self-learners get these wrong

- Using test data for any decision during development — this includes feature engineering
- Not doing EDA before modeling — you will miss data quality issues that ruin models
- Treating feature engineering as less important than algorithm selection — it's the opposite
- Optimizing for accuracy on imbalanced datasets — use F1 or AUC-ROC instead
- Not understanding what a model is actually doing — black-box thinking leads to bad decisions

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Kaggle: Titanic	EDA + 5 algorithms + ensemble + submission	Int	Top 10% ranking, documented feature engineering decisions, algorithm comparison table
Spam Classifier	NLP classification with multiple algorithms	Adv	TF-IDF + 5 algorithms, precision-recall tradeoff analysis, ROC curves, confusion matrices
PH Customer Segmentation	K-Means on Philippine e-commerce dataset	Adv	Optimal K via elbow + silhouette, cluster profiling, actionable business recommendations

House Price Prediction	End-to-end regression with full feature engineering	Adv	Feature importance analysis, log transforms, stacking ensemble, RMSE under 0.13
------------------------	---	-----	---

Self-Check — Can you answer these?

- ☐ Can you explain the bias-variance tradeoff using a diagram you drew yourself?
- ☐ Can you derive gradient descent for linear regression from scratch on paper?
- ☐ Can you explain why target leakage is dangerous and give a real example?
- ☐ Can you describe 5 ways to handle an imbalanced classification dataset with tradeoffs?
- ☐ Can you explain what a kernel does in SVM without using the word 'kernel'?
- ☐ Can you explain when you would choose XGBoost over Random Forest?

When Stuck

If your model is not performing: 80% of the time the problem is the data, not the algorithm. Check: target leakage, data distribution shift between train/test, incorrect label encoding, missing value handling. Plot everything before changing the model.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Andrew Ng ML Specialization	Course	Free	★★★★★	Coursera — audit free, the most important ML course ever made, do everything
Hands-On ML (Geron)	Book	Paid	★★★★★	Best practical ML book — Python, scikit-learn, TF, Keras — read every chapter
Kaggle Learn	Course	Free	★★★★★	Free short courses on ML, feature engineering, data viz — excellent quality
Scikit-learn User Guide	Docs	Free	★★★★★	Best ML library docs — read the user guide, not just the API reference

CS402	Deep Learning & Neural Networks	Free
Duration	14 weeks	Very Hard
Est. Hours	~230 focused hours	
Prereqs	CS401, CS102	

Deep learning is the hardest subject in this curriculum. The math is real, the debugging is painful, and the rewards are extraordinary. Every expert in this field struggled at backpropagation once. Be patient. The breakthrough moments are worth the struggle.

Curriculum Breakdown

Module 1: Neural Network Foundations

- Perceptron: linear classifier, XOR problem, limitations — why we need hidden layers
- Multilayer Perceptron: hidden layers, universal approximation theorem — what it actually means
- Forward pass: matrix multiplications + activation functions — implement in numpy first
- Activation functions: Sigmoid (range 0-1, vanishing gradient), Tanh (range -1 to 1), ReLU (most used), Leaky ReLU, GELU (transformers), Swish
- Loss functions: MSE for regression, Binary CrossEntropy for binary, Categorical for multi-class
- Backpropagation: chain rule applied to computation graphs — DERIVE THIS ON PAPER
- Weight initialization: Xavier/Glorot (sigmoid/tanh), He (ReLU) — why bad init kills training
- Gradient descent: batch (full data), mini-batch (most common), stochastic (noisy but fast)

- Optimizers: SGD (with momentum), RMSProp, Adam (adaptive, most widely used), AdamW (weight decay)

Module 2: Regularization & Training Tricks

- Dropout: randomly zero activations during training — ensembling effect, not for transformers
- Batch normalization: normalize layer inputs, learnable gamma/beta, accelerates training
- Layer normalization: normalize across features not batch — used in Transformers and RNNs
- Early stopping: monitor validation loss, patience, save best weights — most underused trick
- Learning rate schedules: warmup (transformers), step decay, cosine annealing, OneCycleLR
- Gradient clipping: `clip_grad_norm_` — essential for RNNs and training stability
- Weight decay: L2 regularization in optimizer, AdamW does this correctly
- Mixed precision training: float16 for speed, float32 for stability — `torch.autocast`

Module 3: Convolutional Neural Networks

- Convolution operation: filter (kernel), stride, padding, output size formula — compute by hand
- Feature maps: what each filter learns, visualize with Grad-CAM
- Pooling: max pooling (position invariance), average pooling, global average pooling
- Classic architectures: LeNet (concepts), AlexNet (deep learning renaissance), VGG (simplicity), ResNet (skip connections — most important innovation), EfficientNet (scaling)
- Why ResNet works: skip connections prevent vanishing gradient in very deep networks
- Transfer learning: ImageNet features are universal — freeze backbone, train head
- Fine-tuning: unfreeze later layers with low LR — discriminative learning rates
- Data augmentation: RandomHorizontalFlip, RandomCrop, ColorJitter, Mixup, CutMix, AugMix

Module 4: Recurrent Networks & Sequence Modeling

- Why standard NNs fail on sequences: fixed input size, no temporal memory
- Vanilla RNN: hidden state recurrence, BPTT (backprop through time)
- Vanishing gradient in RNNs: why gradients die over long sequences — the fundamental problem
- LSTM: cell state (long-term memory), forget gate, input gate, output gate — draw and understand every gate
- GRU: simplified LSTM — reset gate + update gate, fewer parameters, often comparable performance
- Bidirectional RNN: process sequence in both directions, concatenate hidden states
- Sequence-to-sequence: encoder encodes input, decoder generates output — translation, summarization

Module 5: Transformers & Attention Mechanism

- Attention intuition: when reading, you focus on relevant words — that's attention
- Scaled dot-product attention: Q, K, V matrices, $\text{softmax}(QK^T / \sqrt{d_k}) * V$ — implement from scratch
- Why scale by $\sqrt{d_k}$: prevents softmax saturation in high dimensions
- Multi-head attention: h parallel attention heads, concatenate, project — different aspects of relationships
- Positional encoding: inject sequence position since attention is permutation-invariant
- Transformer encoder: multi-head attention + FFN + residual + LayerNorm — BERT uses this
- Transformer decoder: masked attention (autoregressive) + cross-attention + FFN — GPT uses this
- BERT: masked language modeling (predict masked tokens), NSP, bidirectional context
- GPT family: autoregressive (predict next token), causal masking, in-context learning, emergent abilities
- Vision Transformer (ViT): patch embeddings, position embeddings, transformer encoder on images

Module 6: Generative Models

- Autoencoders: encoder compresses to latent space, decoder reconstructs — learned representation
- Variational Autoencoder (VAE): probabilistic latent space, ELBO loss, reparameterization trick
- GAN: generator fools discriminator, discriminator detects fakes, minimax game, Nash equilibrium
- GAN training instability: mode collapse, vanishing gradients, Wasserstein GAN improvements
- Diffusion models: forward process adds noise, reverse process denoises — the math behind Stable Diffusion
- Score matching and DDPM: the theoretical foundation of modern image generation
- LLM training: pretraining (next token prediction), SFT (instruction following), RLHF (alignment)

Module 7: PyTorch in Depth

- Tensors: creation, indexing, broadcasting rules, device management (`.to('cuda')`), memory layout
- Autograd: computation graph, `requires_grad`, `.backward()`, `.grad`, `detach()`, `no_grad()`
- `nn.Module`: define layers in `__init__`, implement `forward()`, `named_parameters()`, `state_dict()`
- Custom layers: write your own `nn.Module` subclass with learnable parameters

- DataLoader: custom Dataset class, `__len__`, `__getitem__`, transforms, batching, `num_workers`
- Training loop: `zero_grad()`, forward pass, compute loss, `backward()`, `optimizer.step()` — write from scratch
- Evaluation loop: `model.eval()`, `torch.no_grad()`, running metrics — different from training
- Saving: `state_dict()` for weights, full checkpoint for resume training, model export
- PyTorch Lightning: `LightningModule`, `Trainer`, callbacks, logging — cleaner production code
- Debugging: `torch.autograd.set_detect_anomaly`, gradient checking, NaN detection

Common Mistakes — 90% of self-learners get these wrong

- Not implementing backprop from scratch at least once — using autograd without understanding it is a trap
- Not overfitting a single batch first — always verify your model and data pipeline before training fully
- Using the wrong loss function for the task — this is more common than you think
- Not monitoring training curves — loss should decrease smoothly, if it doesn't, debug before waiting for the epoch to finish
- Not normalizing input data — neural networks are very sensitive to input scale

Practice Projects

Project	Description	Lvl	Hireable Version Has...
MNIST from Scratch	Digit classifier, pure PyTorch, no pretrained	Adv	Custom Dataset, DataLoader, training loop, 99%+ accuracy, confusion matrix, Grad-CAM
Transfer Learning	Fine-tune ResNet50 on custom Philippine dataset	Adv	100+ images, 90%+ accuracy, compare frozen vs fine-tuned, Grad-CAM visualization
BERT Sentiment Analysis	Fine-tune BERT on reviews dataset	Adv	HuggingFace Transformers, 93%+ F1, deployed as FastAPI endpoint with rate limiting
Kaggle Deep Learning	Enter one image or NLP competition	Expert	Top 30% finish, public notebook with full explanation, training curves logged to W&B;

Self-Check — Can you answer these?

- ☐ Can you derive backpropagation for a 2-layer network on paper without any reference?
- ☐ Can you explain why ResNet's skip connections solve the vanishing gradient problem — mathematically?
- ☐ Can you explain scaled dot-product attention intuitively, step by step, without formulas?
- ☐ Can you implement a complete training loop in PyTorch from scratch in under 30 minutes?
- ☐ Can you explain what makes BERT different from GPT architecturally and why it matters?
- ☐ Can you explain how diffusion models generate images at a high level?

When Stuck

Loss not decreasing? Step 1: overfit a single batch (set `batch_size=1`). If it converges, your architecture and loss are correct. If not, your forward pass or loss has a bug. Fix that before training on full data. This single debugging technique saves hours.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Andrej Karpathy (YouTube)	Video	Free	★★★★★	makemore, nanoGPT, micrograd — the absolute best teacher for building from scratch
fast.ai Course	Course	Free	★★★★★	Practical Deep Learning — top-down approach, best for building intuition first

deeplearning.ai Specialization	Course	Free	★★★★★	Andrew Ng — 5-course specialization, audit free, foundational
PyTorch Official Tutorials	Docs	Free	★★★★★	pytorch.org/tutorials — official, comprehensive, always up to date

CS403	MLOps & LLM Engineering			Free
Duration	10 weeks			Intermediate-Hard
Est. Hours	~130 focused hours			
Prereqs	CS301, CS401			

A model in a Jupyter notebook is not a product. MLOps is what separates data scientists from ML engineers. LLM Engineering is the hottest skill in the PH tech market in 2025-2026. This subject covers both — because they are now inseparable.

Curriculum Breakdown

ML Pipelines & Experiment Tracking

- Data versioning with DVC: track datasets like code, dvc push/pull, remote storage
- Experiment tracking: MLflow — log params, metrics, artifacts, compare runs, model registry
- Weights & Biases: richer tracking, sweeps for HPO, run comparison, team collaboration
- Reproducibility: random seeds, config files (Hydra), requirements.txt + Python version + model hash
- Feature stores: what they solve, Feast basics — centralized feature computation and serving
- Data validation: Great Expectations, Pandera — validate data contracts before training

Model Deployment Patterns

- Model as REST API: FastAPI endpoint, request batching, async inference
- Model serialization: pickle (fragile, Python only), ONNX (portable, hardware-optimized), TorchScript (PyTorch production)
- Containerizing ML: Docker with model files, handling large model sizes, multi-stage builds
- Cloud deployment: AWS SageMaker endpoints, Google Vertex AI endpoints, Hugging Face Spaces (free)
- Batch vs real-time inference: use cases, cost comparison, latency vs throughput tradeoff
- Model serving frameworks: BentoML (recommended for beginners), TorchServe, Triton Inference Server
- Quantization: int8, float16 — reduce model size by 4x with minimal accuracy loss

Production ML Monitoring

- Data drift: input feature distribution changes over time — Kolmogorov-Smirnov test, PSI
- Concept drift: the relationship between X and y changes — performance metrics decline
- Model performance monitoring: log predictions and ground truth, compute metrics in production
- Evidently AI: open source drift detection and monitoring reports
- A/B testing ML models: shadow deployment first, then traffic split, statistical significance
- Canary deployment: route 5% of traffic to new model, monitor, then increase
- Retraining triggers: scheduled (weekly), performance-based (F1 drops below threshold), drift-based

LLM Engineering — Part 1: Foundations

- Prompt engineering: zero-shot, few-shot, chain-of-thought (CoT), self-consistency
- System prompts: persona, constraints, output format, tone — how to structure them
- Prompt chaining: break complex tasks into a sequence of simpler prompts
- Output parsing: structured output (JSON mode), Pydantic validation of LLM responses
- Token counting: tiktoken, manage context window limits, chunking strategies
- Temperature and top-p: what they control, when to use high vs low temperature
- LLM APIs: OpenAI API, Anthropic API, Google Gemini API — authentication, rate limits, costs

LLM Engineering — Part 2: RAG & Agents

- RAG architecture: retrieve relevant context, augment the prompt, generate with context
- Document processing: PyPDF2, python-docx, unstructured — extract text from any format
- Text chunking: fixed size, recursive, semantic chunking — chunk size and overlap matter

- Embeddings: sentence-transformers (free), OpenAI embeddings, dimensionality and similarity
- Vector databases: FAISS (local, free), ChromaDB (local, free), Pinecone (cloud), Qdrant (self-hosted)
- Retrieval: cosine similarity, MMR (maximal marginal relevance) for diversity, hybrid search
- LangChain: chains, LCEL (LangChain Expression Language), retrievers, memory
- LlamaIndex: nodes, indexes, query engines — alternative to LangChain for document Q&A;
- Agents and tools: ReAct pattern, tool calling (function calling), multi-step reasoning
- Evaluation: RAGAS (faithfulness, relevance, context recall), LLM-as-judge, human eval

LLM Engineering — Part 3: Fine-Tuning & Production

- When to fine-tune vs RAG: RAG for knowledge, fine-tuning for style/format/behavior
- LoRA: Low-Rank Adaptation — fine-tune with 1% of parameters, full fine-tune quality
- QLoRA: LoRA + 4-bit quantization — fine-tune 7B models on a single GPU
- PEFT library: LoRA, QLoRA, prefix tuning — HuggingFace ecosystem
- Training data preparation: instruction following format, data quality > quantity
- Evaluation after fine-tuning: compare against base model, check for catastrophic forgetting
- Multi-agent systems: orchestrator + specialist agents, tool use, reflection, planning
- Building a production LLM app: streaming responses, error handling, fallbacks, cost tracking

Common Mistakes — 90% of self-learners get these wrong

- Jumping to fine-tuning when RAG would solve the problem better and cheaper
- Not chunking documents properly — bad chunking is the most common RAG failure mode
- Not evaluating LLM outputs systematically — vibe-checking is not evaluation
- Ignoring cost tracking — LLM APIs can get expensive very fast in production
- Not handling LLM failures (timeouts, rate limits, hallucinations) gracefully in production

Practice Projects

Project	Description	Lvl	Hireable Version Has...
ML Production API	Trained model as FastAPI endpoint with full MLOps	Adv	MLflow tracking, Docker, versioned endpoints, drift monitoring, auto-retraining pipeline
RAG Document System	Q&A; over your own document collection	Adv	Chunking comparison, retrieval metrics, citation in answers, RAGAS evaluation, deployed
LLM Agent	Multi-step agent that completes real tasks	Adv	Custom tools, ReAct loop, streaming, error recovery, cost tracking, deployed as web app
Fine-Tuned Model	QLoRA fine-tune a 7B model for a specific task	Expert	Training data prep, PEFT training, evaluation vs base model, pushed to HuggingFace Hub

Self-Check — Can you answer these?

- ☐ Can you explain the difference between data drift and concept drift with real examples?
- ☐ Can you describe the RAG architecture completely — from document to answer?
- ☐ Can you explain when fine-tuning is better than RAG and when RAG is better than fine-tuning?
- ☐ Can you set up MLflow experiment tracking for a training run from scratch?
- ☐ Can you implement a ReAct agent with custom tools using LangChain or from scratch?
- ☐ Can you explain what RAGAS measures and how to interpret the scores?

When Stuck

RAG not returning relevant chunks? The problem is almost always: (1) bad chunking — try smaller chunks with overlap, (2) wrong embedding model — try a domain-specific one, (3) retrieval k too small — try k=10 then rerank. Measure retrieval precision before blaming generation.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Made With ML (Goku)	Course	Free	★★★★★	madewithml.com — best free end-to-end ML engineering course
LangChain Docs	Docs	Free	★★★★★	python.langchain.com — comprehensive but verbose, use with cookbook examples
Designing ML Systems (Huyen)	Book	Paid	★★★★★	The definitive guide to production ML — read every chapter
HuggingFace PEFT Docs	Docs	Free	★★★★★	huggingface.co/docs/peft — official LoRA/QLoRA documentation

CS404	ML Specialization (Pick One Track)	Free
Duration	8 weeks	Hard
Est. Hours	~120 focused hours	
Prereqs	CS402	

After mastering foundations, go deep in one area. For the PH market in 2025-2026, NLP/LLM has the most demand followed by Computer Vision. Choose based on what genuinely excites you — you will go further in an area you love than one you tolerate.

Curriculum Breakdown

Track A: NLP & LLM Engineering (Recommended for PH market)

- Text preprocessing: tokenization (word, character, subword), BPE, WordPiece, SentencePiece
- Classic embeddings: Word2Vec (CBOW, Skip-gram), GloVe, FastText — understand before using transformers
- HuggingFace Transformers: from_pretrained, AutoModel, AutoTokenizer, Trainer API
- Text classification: fine-tune BERT/DistilBERT, multi-label classification, label smoothing
- NER: token classification, BIO tagging scheme, evaluation with seqeval
- Question Answering: extractive (SQuAD format), abstractive (seq2seq)
- Summarization: extractive (TextRank), abstractive (BART, T5), ROUGE evaluation
- Translation: MarianMT, Helsinki-NLP models, BLEU evaluation
- Building LLM applications: structured output, function calling, streaming, multi-turn conversation
- LLM evaluation: benchmark datasets, human eval, LLM-as-judge, A/B testing

Track B: Computer Vision

- Image preprocessing pipelines: torchvision transforms, augmentations (more powerful)
- Object detection: YOLO family history, YOLOv8 (Ultralytics) — train on custom dataset
- Object detection metrics: mAP, IoU, precision-recall curve at different thresholds
- Image segmentation: semantic (U-Net), instance (Mask R-CNN), panoptic
- Segment Anything Model (SAM): zero-shot segmentation, prompts, applications
- Facial recognition: face detection (MTCNN), embeddings (ArcFace), comparison pipeline
- Video understanding: optical flow, temporal convolutions, slow-fast networks overview
- Deployment optimization: TensorRT (NVIDIA), ONNX Runtime, quantization for edge devices
- OpenCV: image processing, geometric transforms, feature detection (ORB, SIFT), template matching

Track C: Reinforcement Learning

- MDP formulation: states S , actions A , rewards R , policy π , value function V , Q-function
- Bellman equations: the recursive definition of value — derive and understand
- Q-Learning: tabular, convergence proof, exploration-exploitation (epsilon-greedy)
- Deep Q-Network (DQN): experience replay buffer, target network, double DQN, dueling DQN
- Policy gradient: REINFORCE algorithm, baseline subtraction, variance reduction
- Actor-Critic: A2C (synchronous), A3C (asynchronous), advantage function

- PPO (Proximal Policy Optimization): clipped objective, trust region, most stable algorithm
- SAC (Soft Actor-Critic): entropy maximization, continuous action spaces, off-policy
- OpenAI Gymnasium: CartPole, LunarLander, MuJoCo (physics simulation)
- Applications: game agents, robotic control, recommendation systems, LLM alignment (RLHF)

Common Mistakes — 90% of self-learners get these wrong

Trying to do all three tracks — pick one and go deep, breadth comes later

Not collecting your own dataset — a model trained on your own data is more impressive

Evaluating with the wrong metrics — don't use accuracy for object detection

Not publishing your project — push to HuggingFace Hub or GitHub with a live demo

Practice Projects

Project	Description	Lvl	Hireable Version Has...
NLP: Full Document QA	Multi-document Q&A; with HuggingFace + RAG	Expert	Handles 100+ documents, cites sources, deployed with React frontend, RAGAS evaluated
CV: YOLOv8 Detector	Train on custom Philippine dataset	Expert	85%+ mAP, real-time demo, deployed as mobile-friendly web app, data collection documented
RL: Multi-Task Agent	PPO agent mastering 3 Gymnasium environments	Expert	Training curves in W&B;., video recordings, reward shaping documented, hyperparameter study

Self-Check — Can you answer these?

- ☐ Can you fine-tune a HuggingFace model end-to-end on a custom dataset?
- ☐ Can you explain the difference between BLEU and ROUGE and when each is appropriate?
- ☐ Can you explain why PPO is more stable than vanilla policy gradient — mathematically?
- ☐ Can you train a YOLOv8 model on a custom dataset and evaluate its mAP?

When Stuck

When fine-tuning feels overwhelming: start with the smallest model in the family (DistilBERT, not BERT-large; YOLOv8n, not YOLOv8x). Get it working first. Scale up only after you have a working baseline. Bigger models do not always mean better results.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
HuggingFace Course	Course	Free	★★★★★	huggingface.co/learn — best free NLP/LLM course, very hands-on
Ultralytics YOLOv8	Docs	Free	★★★★★	docs.ultralytics.com — excellent docs, train in 10 lines of code
Spinning Up in DRL	Course	Free	★★★★★	spinningup.openai.com — OpenAI's RL resource, mathematical but accessible
Papers With Code	Website	Free	★★★★★	paperswithcode.com — state-of-the-art with code, find datasets and baselines

CS405

Data Visualization & Communication

Free

Duration 3 weeks

Intermediate

Est. Hours ~40 focused hours

Prereqs CS401

Half the job of any ML or data role is communicating your findings to people who don't understand the math. A model that isn't understood won't be trusted. A finding that isn't visualized clearly won't be acted on. This subject makes your technical work land.

Curriculum Breakdown

Static Visualization

- matplotlib: figure, axes, subplots, figure size, DPI, saving — the foundation
- Plot types: line, bar, scatter, histogram, box, violin, heatmap — when each is appropriate
- matplotlib styling: rcParams, stylesheets, color maps, figure aesthetics
- seaborn: statistical visualization on top of matplotlib, pairplot, FacetGrid, categorical plots
- Pandas plotting: quick EDA plots, .plot() API, directly from DataFrames
- Visualization principles: data-ink ratio, lie factor, color perception, preattentive attributes
- Color: accessible palettes (ColorBrewer), sequential vs diverging vs qualitative, colorblind safe

Interactive Visualization

- Plotly: interactive charts, hover info, animations, export to HTML — best for dashboards
- Plotly Express: high-level API, one-line charts — use for EDA
- Plotly Graph Objects: full control — use for production dashboards
- Dash: full dashboards in Python — callback functions, layout, components
- Streamlit: fastest way to build ML demos — perfect for portfolio projects
- Gradio: ML model demos with automatic UI — ideal for HuggingFace Spaces deployment
- Tableau Public: drag-and-drop BI tool — free tier sufficient, learn the basics

Communicating ML Results

- Executive summary: one paragraph, no jargon, what the model does, what it improves
- Model card: intended use, limitations, evaluation metrics, bias analysis, how to use
- Confusion matrix: visualize and explain to non-technical stakeholders
- Feature importance plots: SHAP values (SHapley Additive exPlanations) — explain individual predictions
- Learning curves: communicate model progress and what would improve it
- A/B test results: statistical significance, confidence intervals, business impact — present honestly
- Presentation structure: problem, data, approach, results, limitations, next steps

Common Mistakes — 90% of self-learners get these wrong

- Using 3D pie charts or any chart that distorts perception — never
- Showing accuracy metrics to business stakeholders without explaining what they mean
- Not visualizing your model's failures — where it goes wrong is as important as overall metrics
- Ignoring colorblind accessibility — 8% of men have color vision deficiency

Practice Projects

Project	Description	Lvl	Hireable Version Has...
EDA Dashboard	Streamlit dashboard for a Kaggle dataset	Int	Interactive filters, multiple chart types, summary statistics, download report button
ML Explainability Report	SHAP analysis for your CS401 house price model	Int	Global feature importance, individual prediction explanation, partial dependence plots
Model Card	Write a complete model card for your best CS402 model	Int	Follows HuggingFace model card format, includes bias analysis and limitations

Self-Check — Can you answer these?

- ☐ Can you explain what SHAP values represent to a non-technical business stakeholder?
- ☐ Can you build an interactive Streamlit dashboard for a model in under 2 hours?

- Can you identify 3 things wrong with a poorly designed chart when shown one?
- Can you write a one-paragraph executive summary of a model's performance and value?

When Stuck

If your chart is confusing: remove half of it. Add a descriptive title that states the finding, not just the variable names. Good charts have one clear message. If your chart needs a long explanation, redesign the chart.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Matplotlib Documentation	Docs	Free	★★★★ ■	matplotlib.org — comprehensive, use as reference
Streamlit Documentation	Docs	Free	★★★★ ★	docs.streamlit.io — excellent docs, get an app running in 30 minutes
SHAP Library	Docs	Free	★★★★ ★	shap.readthedocs.io — explainability for any ML model, critical skill
Storytelling with Data	Book	Paid	★★★★ ★	Cole Knaflic — best book on data visualization communication

CS406	Data Engineering Foundations	Free
Duration	6 weeks	Intermediate-Hard
Est. Hours	~80 focused hours	
Prereqs	CS203, CS301	

Many 'ML Engineer' job postings in the Philippines are actually Data Engineering roles in disguise. Even pure ML roles require data engineers upstream. This subject makes you T-shaped: strong ML foundations with enough data engineering to collaborate effectively — and enough to get data engineering roles as an alternative path.

Curriculum Breakdown

Data Pipeline Fundamentals

- ETL vs ELT: extract-transform-load vs extract-load-transform — the modern shift to ELT
- Batch processing: process large datasets periodically — nightly jobs, weekly reports
- Stream processing: process data as it arrives — real-time dashboards, fraud detection
- Data pipeline orchestration: Airflow (the standard), Prefect (modern, Python-native), Dagster
- Apache Airflow: DAGs, operators, sensors, connections, XComs, the Airflow UI
- Pipeline testing: unit test transforms, integration test pipelines, data quality checks in-pipeline

Modern Data Stack

- Data warehouses: OLAP vs OLTP — optimized for analytics vs transactions
- BigQuery: serverless, columnar, SQL interface, partitioning, clustering, cost control
- Snowflake: virtual warehouses, time travel, zero-copy cloning, shares — conceptual overview
- dbt (data build tool): models, tests, documentation, lineage graph — SQL-based transformations
- dbt project structure: staging, intermediate, marts — the layered approach
- Data quality: dbt tests (unique, not_null, relationships, accepted_values), custom tests
- Data catalog: understanding metadata, column lineage, schema evolution

Big Data Processing

- Why big data tools: when pandas fails (> RAM size), distributed computing
- Apache Spark: RDD, DataFrame API, SparkSQL — Python (PySpark) interface

- Spark concepts: driver, executors, partitions, shuffles — why shuffles are expensive
- PySpark: read CSV/Parquet/JSON, transformations, actions, writing to cloud storage
- Parquet format: columnar storage, compression, predicate pushdown — why it's better than CSV
- Delta Lake: ACID transactions on data lakes, time travel, schema enforcement
- Kafka: producers, consumers, topics, partitions, consumer groups — stream ingestion

Data Engineering in Practice

- Data modeling: star schema (fact + dimension tables), snowflake schema — dimensional modeling
- Slowly Changing Dimensions (SCD): Type 1 (overwrite), Type 2 (history), Type 3 (previous value)
- Data lake architecture: raw layer, cleaned layer, curated layer — bronze/silver/gold
- Data mesh concepts: domain ownership, data as a product, self-serve infrastructure
- Cloud data services: BigQuery, AWS Redshift, Azure Synapse — know what each is
- Data engineering interview: SQL window functions, system design (design a data pipeline), Python data manipulation

Common Mistakes — 90% of self-learners get these wrong

Using pandas for data that doesn't fit in RAM — learn when to graduate to Spark

Not testing data pipelines — a pipeline that silently produces wrong results is worse than one that fails

Not handling schema evolution — upstream changes break pipelines if not planned for

Over-engineering small pipelines — a cron job + Python script is often sufficient

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Airflow ETL Pipeline	Daily pipeline: API > transform > load to BigQuery	Int	Retries on failure, data quality checks, email alert on failure, dashboard in Streamlit
dbt Project	Transform raw e-commerce data into analytics-ready models	Int	Staging, intermediate, mart layers, all tests passing, documented with dbt docs generate

Self-Check — Can you answer these?

- ☐ Can you explain the difference between ETL and ELT and when each is appropriate?
- ☐ Can you write a PySpark job that reads Parquet, transforms data, and writes to Cloud Storage?
- ☐ Can you explain the star schema and why it's preferred for analytics workloads?
- ☐ Can you design a simple Airflow DAG for a daily data pipeline with error handling?

When Stuck

If a Spark job is slow: look for shuffles (sortBy, groupBy, join). Shuffles move data across the network — they're expensive. Repartition before joins, use broadcast joins for small tables, and cache DataFrames you reuse.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
Fundamentals of Data Eng. (Reis)	Book	Paid	★★★★★	The definitive data engineering book — read chapters 1-6 minimum
dbt Documentation	Docs	Free	★★★★★	docs.getdbt.com — excellent, with a free cloud version to practice on
BigQuery Documentation	Docs	Free	★★★★★	cloud.google.com/bigquery — free \$300 credit to experiment
Apache Airflow Docs	Docs	Free	★★★★★	airflow.apache.org — comprehensive, use Astronomer's tutorials to start

Phase Capstone

Build a complete ML product end-to-end: a Filipino language sentiment analysis API for social media posts. Data: collect and label 5000+ Filipino tweets. Pipeline: Airflow ETL to BigQuery, dbt transformations, feature engineering. Model: fine-tune a multilingual BERT (mBERT or XLM-RoBERTa) with LoRA, tracked in MLflow, served via FastAPI on Cloud Run, monitored with Evidently AI. Frontend: Streamlit dashboard with SHAP explanations and Plotly visualizations. RAG system: answer questions about Filipino social media trends using your own data. This is your capstone portfolio piece — the one you talk about in every interview.

PHASE 5

~200 hrs

Career & Interview Preparation

Skills mean nothing if you cannot demonstrate them under pressure. Get hired. Know your worth.

Est. Duration	Ongoing — intensify 3 months before target start date	Est. Total Hours	~200 hours
----------------------	---	-------------------------	-------------------

Must Complete (MVP)	Nice-to-Have (If Time Allows)
• CS501 Technical interviews (DSA + system design)	• Open source contributions
• CS502 Portfolio + job search strategy	• Technical blog with 1000+ monthly readers
	• Conference talks

Interview Habit

Note: if you've been following the interview habits from each phase, you've already done 12+ mock interviews and 40+ Codeforces contests by the time you reach Phase 5. You are already prepared. This phase is refinement, not starting from zero.

CS501	Technical Interview Mastery	Free
Duration	Intensify 3 months before applying	Hard
Est. Hours	~120 focused hours	
Prereqs	CS201	

Technical interviews are a skill of their own. The best engineers fail them because they've never practiced the format. The goal is not to be the smartest — it is to be the most practiced. Deliberate repetition beats raw intelligence every time.

Curriculum Breakdown

Coding Interview Framework

- UCTPV: Understand the problem fully, Clarify edge cases, Think aloud (never go silent), Plan the approach, Verify with test cases
- Clarify before coding: input constraints, edge cases (empty, single element, max size), expected output format
- Think aloud always: narrate your thought process — silence is interpreted as being stuck
- Brute force first: state the naive $O(n^2)$ or $O(2^n)$ solution before optimizing — shows structured thinking
- Optimize with questions: 'can I use extra space?', 'is the input sorted?', 'what are the constraints?'
- Test manually: trace through 2-3 examples including edge cases before saying you're done
- Analyze complexity: always state time and space complexity when you finish — don't wait to be asked

The 10 Coding Patterns (Master These)

- Sliding Window: substring/subarray with size constraint — 2 pointers that expand/shrink
- Two Pointers: sorted array, palindrome check, 3-sum — works when order matters
- Binary Search: sorted array, rotated array, answer-range search (minimize max, maximize min)
- BFS/DFS: graphs, trees, grids — BFS for shortest path, DFS for existence/backtracking
- Dynamic Programming: overlapping subproblems + optimal substructure — 5 problem families

- Backtracking: generate all valid combinations — subsets, permutations, N-queens, word search
- Heap/Priority Queue: top K elements, K closest points, median of stream, scheduling
- Intervals: merge overlapping, insert interval, meeting rooms — sort by start time first
- Linked List: fast/slow pointer, reverse in k-groups, merge k sorted lists
- Monotonic Stack: next greater element, daily temperatures, largest rectangle in histogram

System Design Interview Framework

- Total time: 45 minutes — budget it explicitly at the start
- Clarify (5 min): functional requirements (what it does), non-functional (scale, latency, availability, consistency)
- Estimate (5 min): DAU, QPS (reads + writes), storage (per day, per year), bandwidth — use round numbers
- High-level design (10 min): 3-5 boxes with arrows, name each component, explain data flow
- Deep dive (20 min): interviewer picks 1-2 areas — go into detail, present tradeoffs explicitly
- Wrap-up (5 min): identify bottlenecks, discuss what you would improve with more time
- Always state tradeoffs: 'I chose X over Y because... the downside of X is...'

Behavioral Interviews (STAR Method)

- Situation: 1-2 sentences, enough context for the interviewer to understand the stakes
- Task: what was YOUR specific responsibility — not the team's, yours
- Action: what YOU specifically did — concrete, step-by-step, not vague
- Result: quantify if possible (reduced latency by 40%, increased conversion by 12%) — business impact
- Prepare 10 stories covering: leadership, technical decision, conflict with teammate, failure/learning, ambiguous situation, mentoring, under pressure, proactively identifying a problem, disagreeing with manager, most proud achievement
- Common questions: tell me about yourself (2 min), greatest technical challenge, time you failed, disagreement with manager, why this company
- Research: read their engineering blog, recent news, tech stack (LinkedIn, Glassdoor, tech radar)

Common Mistakes — 90% of self-learners get these wrong

Grinding 300 LeetCode problems of the same type instead of learning all 10 patterns

Practicing coding in silence — the interviewer needs to hear your thinking

Not practicing system design until Phase 5 — you should have done monthly mocks from Phase 3

Being too proud to ask for hints — asking clarifying questions is professional, not weak

Not researching the company — interviewers notice when you don't know the product

Practice Projects

Project	Description	Lvl	Hireable Version Has...
NeetCode 150	Complete the full organized problem set	Adv	All solved, pattern identified, time/space complexity documented, optimal solutions understood
20 Mock Interviews	Pramp, Interviewing.io, or with peers	Adv	Record yourself, review communication, note where you went silent or confused
7 System Design Walkthroughs	Solo timed 45-min design sessions	Adv	Draw on paper, present tradeoffs, review against ByteByteGo solutions
Behavioral Stories Bank	Written STAR stories for 10 scenarios	Adv	Practiced out loud, timed, refined based on feedback, ready to deliver in 2 minutes each

Self-Check — Can you answer these?

- ☐ Can you solve any Medium DP problem you've never seen before in under 35 minutes?
- ☐ Can you design a URL shortener from scratch in 45 minutes with explicit tradeoffs?
- ☐ Can you deliver your 'tell me about yourself' answer in exactly 90 seconds?
- ☐ Can you name all 10 coding patterns and immediately give 2 example problems for each?
- ☐ Can you identify which pattern applies to a new LeetCode problem in under 2 minutes?

When Stuck

If you freeze during an interview: say 'Let me think through this step by step.' Then restate the problem in your own words. This buys time, shows composure, and often reveals the solution. Never go silent for more than 30 seconds without narrating your thinking.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
neetcode.io	Platform	Free	★★★★★	Best organized roadmap with video solutions — use as primary resource
Pramp.com	Platform	Free	★★★★★	Free live mock interviews with real engineers — do 20+ sessions
Tech Interview Handbook	Website	Free	★★★★★	techinterviewhandbook.org — resume, behavioral, coding all in one
Cracking the Coding Interview	Book	Paid	★★★★★	McDowell — read the mindset chapters, not just the problems

CS502	Portfolio, Brand & Job Search Strategy	Free
Duration	Build continuously from Phase 1	Medium
Est. Hours	~80 focused hours	
Prereqs	CS301, CS302	

In the Philippines, your GitHub, LinkedIn, and portfolio website are reviewed before you get a single phone call. Your online presence is your resume before your resume. Build in public from day one — every week, every project, every learning. This is the fastest path to inbound opportunities.

Curriculum Breakdown

Building a Standout Portfolio

- Quality over quantity: 3 exceptional projects beat 10 mediocre ones — curate ruthlessly
- Every project must have: live deployed demo (not localhost), GitHub link with clean code, professional README
- README structure: what it does (one paragraph), why you built it, tech stack with badges, key features, how to run it, screenshots/GIF
- Case study per project (200-400 words): problem → approach → technical decisions → what you'd do differently
- The capstone projects from each phase ARE your portfolio — build them to production quality
- Deployed projects are non-negotiable: Vercel (frontend), Railway/Render (backend), Hugging Face Spaces (ML models)

GitHub Profile Mastery

- Profile README: pinned at top, who you are, what you build, current focus, contact
- Write it like a landing page, not a resume — make someone want to hire you in 30 seconds
- Pinned repositories: only your 6 best — remove class projects and tutorial code
- Consistent commit activity: daily commits signal consistent work habit to recruiters
- Write meaningful commit messages — recruiters DO read them in standout candidates
- Contribute to real open source: even bug fixes in docs count — shows collaboration skills
- GitHub Actions on all projects: green CI badges signal professional code quality

LinkedIn & Online Presence Strategy

- Headline formula: [Role] | [Top Skills] | [What you build] — not your current job title
- Example: 'Full-Stack + ML Engineer | FastAPI, React, PyTorch | Building AI-powered products'
- About section: 3 paragraphs — your story, your technical strengths, what you're looking for
- Featured section: pin your portfolio site, your best live demo, your best technical article
- Skills: add real skills (Python, FastAPI, React, PyTorch), get peers to endorse them

- Content strategy: one technical post per week — what you learned, what you built, what surprised you
- PH tech communities: Developers Connect PH, Python Philippines, ML Philippines Facebook groups

Philippines Job Market Strategy

- Entry-level targets: Exist Global, Pointwest, Accenture PH, ING Hubs PH, KMC Solutions
- Product companies: GCash (Mynt), Paymaya (Voyager), Kumu, Sprout Solutions, PayMongo
- Multinational tech: Booking.com Manila, Amazon AWS Manila, Google Philippines, Grab Philippines
- Remote-first international: Toptal (highest bar, highest pay), Andela, X-Team, Deel network, Remote.com
- FAANG+: prepare 6 months specifically, mock interview 3x per week — they hire here
- Salary benchmarks 2025: Junior Dev P40-70K, Mid Dev P80-120K, Senior Dev P150-250K
- Remote international: \$2,000-5,000/month USD — 3-5x local rates for equivalent skills
- Freelance ramp: Upwork (build reputation with small projects), then rate increase, then direct clients

Salary Negotiation (The PH-Specific Script)

- Never give a number first: 'I'd like to understand the full package before discussing numbers'
- Research first: Glassdoor, LinkedIn Salary, levels.fyi, asking peers in the community
- When pushed for a number: give a range based on your research, put your target at the bottom
- Counter-offer script: 'Thank you for the offer. Based on my research and experience, I was expecting [X]. Is there flexibility?'
- Never accept on the spot: 'I'd like 48 hours to review the full offer letter — is that okay?'
- Total compensation: base, 13th month, HMO, stock/equity, performance bonus, learning budget, remote work
- The silence tactic: state your counter-offer, then stop talking — whoever speaks first loses leverage
- Walk away power: always have a competing offer or your current income as a floor — it's real leverage

Common Mistakes — 90% of self-learners get these wrong

- Waiting to apply until everything feels ready — start applying at Phase 3, not Phase 5
- Putting your expected salary too low because you're afraid — research the market first
- Not following up after applications — one polite follow-up email after 5 business days is professional
- Treating the job search as passive — you need to warm up connections, not just click apply
- Accepting the first offer immediately — always take 24-48 hours and always counter

Practice Projects

Project	Description	Lvl	Hireable Version Has...
Portfolio Website	Custom-built site, zero templates	Adv	Mobile-first, dark mode, WCAG AA, Lighthouse 90+, project case studies, blog, contact form
10 Technical Blog Posts	Publish on dev.to or your own site	Int	At least 2 posts with 500+ views — document your capstone projects, your learnings
Job Campaign	Apply to 60 companies, track everything	Adv	20 warm applications via network, follow-up emails drafted, spreadsheet with full history

Self-Check — Can you answer these?

- ☐ Do you have 3 projects deployed live with live demo URLs right now?
- ☐ Can you explain any of your portfolio projects in 3 minutes with clear technical depth?
- ☐ Is your GitHub showing consistent daily commits for the past 3+ months?
- ☐ Have you researched and written down your target salary range with sources to back it up?
- ☐ Can you deliver your salary counter-offer without hesitating or apologizing?

When Stuck

Getting no callbacks? 90% of the time the problem is your resume or portfolio — not the market. Send your resume to 3 senior engineers you trust and ask for brutally honest feedback. The feedback will be uncomfortable and invaluable.

Resources (starred by quality)

Resource	Type	Cost	Stars	Notes
levels.fyi	Website	Free	★★★★★	Tech salary data globally — know your worth before every negotiation
Tech Interview Handbook	Website	Free	★★★★★	Job search strategy, resume templates, negotiation scripts — read fully
Developers Connect PH	Community	Free	★★★★★	Facebook group — active PH dev community, jobs posted, salary discussions
LinkedIn	Platform	Free	★★★★★	Most important professional platform in PH — optimize your profile before applying

Phase Capstone

This phase has no technical capstone — you are the capstone. By this point you have: 4 deployed production projects (one per phase), 10+ technical blog posts, 20+ mock interviews completed, NeetCode 150 solved, 7 system design walkthroughs done, a professional GitHub profile with daily activity, and a LinkedIn optimized for inbound. The only thing left is sending the applications. Send 5 today.

Philippines Tech Career Guide 2025

This section is specific to your context. Use the global curriculum above as your foundation. Use this as your local execution strategy.

Salary Benchmarks — Metro Manila, 2025 Estimates

Role	Local Company	Multinational PH	Remote USD/mo	Remote PHP equiv.
Junior Dev (0-2 yrs)	P40-65K	P60-90K	\$1,000-2,000	P57K-114K
Mid Dev (2-4 yrs)	P80-120K	P110-160K	\$2,500-4,500	P143K-257K
Senior Dev (4+ yrs)	P140-220K	P200-300K	\$5,000-9,000	P286K-514K
ML Engineer (entry)	P65-95K	P95-150K	\$2,000-4,000	P114K-228K
ML Engineer (mid)	P120-180K	P170-250K	\$4,500-8,000	P257K-457K
Data Engineer (mid)	P110-160K	P150-220K	\$4,000-7,000	P228K-400K
Cloud/DevOps Eng. (mid)	P100-150K	P140-200K	\$3,500-7,000	P200K-400K

PH Context

The fastest path to financial freedom in the Philippines tech industry: build strong fundamentals (Phases 1-3), then target remote international companies. A mid-level remote role at \$3,500/month = P200,000+ — that is 2-3x the local rate for identical skills. Platforms: Toptal (highest bar, highest pay — apply after Phase 3), Andela, X-Team, Deel network, Remote.com, Upwork (freelancing to build remote track record). The certification that unlocks remote roles fastest: AWS Solutions Architect Associate.

Final Words

You started this journey saying you regret not going to a prestigious university. Here is the truth no one tells you: the school on your diploma matters far less than what you have built and what you can demonstrate under pressure. The engineers who change the world — and the ones who earn the most — got there by building obsessively, not by attending the right lectures.

This curriculum is the equivalent of — and in several areas exceeds — what is taught in the top CS programs in the Philippines and the world. The only difference is that no one will push you. No grades, no deadlines imposed from outside, no classmates to keep pace with. Only you and the editor and the compiler and the error message.

You mentioned you can solve 800-rated problems on Codeforces. That is a real foundation — not nothing. By the time you finish Phase 2 of this curriculum, 1400 will feel like 800 feels today. By Phase 5, you will solve problems in interviews that you currently cannot imagine solving.

The gap between where you are and where you want to be is not talent. It is not school prestige. It is not connections. It is hours of focused work — accumulated, compounding, irreversible. Start Phase 1 today. Code every day. Build every week. Ship every month. Write about what you learn. In two years, you will not recognize your own capabilities.

Must Do

The most important habit in this entire curriculum: open your code editor before you open any other application every morning. Even for 15 minutes. Even if nothing works. The discipline of showing up is the skill that compounds the fastest.